CHAPITRE IV

IV. Fonctions d'affichage

Les premiers *PC* sont apparus équipés de cartes d'affichage MDA (*Monochrome Display Adapter*), autorisant l'affichage en mode texte de 25 lignes de 80 caractères, suivis aussitôt de l'introduction d'un mode d'affichage dit "haute résolution" (CGA *Color Graphics Adapter*) graphique (640 par 200 pixels). Parallèlement ont été introduites les cartes Hercules graphiques (348 lignes de 720 pixels) qui n'ont jamais été officiellement soutenues par IBM.

L'évolution suivante, avec l'introduction des *PC/AT*, a vu l'apparition des cartes EGA (*Enhanced Graphics Adapter*), texte et graphique de 350 lignes de 640 pixels, et plus éphémèrement des cartes PGA (*Professionnal Graphics Adapter*).

La gamme des PS/2 est apparue équipée de cartes MCGA (*MultiColor Graphics Array*) ou VGA (*Video Graphics Array*). Ce dernier mode d'affichage est maintenant proposé de façon systématique sur tous les *PC* et compatibles.

Un groupe de constructeurs de cartes d'affichage (VESA *Video Electronics Standards Association*) a introduit un nouveau standard nommé *Super VGA*. Ce standard définit un ensemble matériel + logiciel (architecture et définition d'appels au BIOS).

Les définitions les plus hautes (1024 x 768) sont fournies par les modes d'affichage IBM 8514/A et XGA, et TIGA (*Texas Instruments Graphics Architecture*).

N°	Mode	Coul.	Texte	Segment	Définition	Pages	Supporté par les modes d'affichage :
0/1	AN	16	40x25	B800	320x200	8	CGA, MCGA, EGA, VGA
					320x350	8	EGA, MCGA, VGA
					320x400	8	MCGA
					360x400	8	VGA
2/3	AN	16	80x25	B800	640x200	4	CGA
					640x200	8	EGA, MCGA, VGA
					640x350	8	EGA, MCGA, VGA
					640x400	8	MCGA
					720x400	8	MCGA, VGA

4/5	GR	4	40x25	B800	320x200	1	CGA, EGA, MCGA, VGA
6	GR	2	80x25	B800	640x200	1	CGA, EGA, MCGA, VGA
7	AN	mono	80x25	B000	720x350	1	MDA
					720x350	8	EGA, MCGA, VGA
					720x400	8	MCGA, VGA
					640x200	4	
8	GR	16	20x25	B800	160x200	1	PC-Junior
9	GR	16	40x25	B800	320x200	1	PC-Junior
A	GR	4	80x25	B800	640x200	1	PC-Junior
D	GR	16	40x25	A000	320x200	8	EGA, MCGA, VGA
E	GR	16	80x25	A000	640x200	4	EGA, MCGA, VGA
F	GR	mono	80x25	A000	640x350	4	EGA, MCGA, VGA
10	GR	16	80x25	A000	640x350	2	EGA, MCGA, VGA
11	GR	2	80x30	A000	640x480	2	MCGA, VGA
12	GR	16	80x30	A000	640x480	1	VGA
13	GR	256	40x25	A000	320x200	1	MCGA, VGA
14	GR	16			640x200		EGA/VGA
15	GR	mono			640x350		EGA/VGA
16	GR	4/16			640x350		EGA/VGA
17	GR	2			640x480		VGA
18	GR	16			640x480		VGA
19	GR	256			320x200		VGA (1 octet par pixel)
30					720x350		3270
6A	GR	16			800x600		Super VGA (≈mode 12)
100	GR	256			640x400		Super VGA 1.0
101	GR	256			640x480		Super VGA 1.0
102	GR	16			800x600		Super VGA 1.0
103	GR	256			800x600		Super VGA 1.0
104	GR	16			1024x768		Super VGA 1.0
105	GR	256			1024x768		Super VGA 1.0
106	GR	16			1280x1024		Super VGA 1.0
107	GR	256			1280x1024		Super VGA 1.0
108	AN		80x60				Super VGA 1.1
109	AN		132x25				Super VGA 1.1
10A	AN		132x43				Super VGA 1.1
10B	AN		132x50				Super VGA 1.1
10C	AN		132x60				Super VGA 1.1

6A	GR	16			800x600		VESA
100	GR	256			640x400		VESA
101	GR	256			640x480		VESA
102	GR	16			800x600		VESA
103	GR	256			800x600		VESA
104	GR	16			1024x768		VESA
105	GR	256			1024x768		VESA
106	GR	16			1280x1024		VESA
107	GR	256			1280x1024		VESA

L'interruption 10H du BIOS permet de gérer ces différents modes d'affichage.

IV.1. Les fonctions de l'interruption INT 10H

IV.1.1. Les fonctions standards

INT 10H/0 — Positionnement mode vidéo

AH 0
AL mode vidéo

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
BIOS EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS ne sont pas modifiés.
Si le bit 7 du mode vidéo est mis à 1, la mémoire n'est pas effacée.

INT 10H/1 — Positionnement attributs curseur

AH 1
CH ligne de trame de début pour le curseur :
– bit 7 : curseur visible ou invisible
– bits 6-0 à 0 si curseur normal
– bits 6-0 à 1 si clignotement modifié
CL ligne de trame de fin pour le curseur

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/2 — Positionnement curseur

AH 2
DH ligne
DL colonne
BH numéro de page d'affichage

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/3 — Acquisition position et attributs du curseur

AH 3
BH numéro de page d'affichage

DH ligne
DL colonne
CH,CL attributs du curseur

BX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/5 — Sélection page active (page de texte seulement)

AH 5
AL nouvelle page

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/6 — Défilement vers le haut

AH 6
CH,CL ligne, colonne coin haut gauche
DH,DL ligne, colonne coin bas droit
BH attribut lignes insérées
AL nombre de lignes (0 pour effacer)

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.
Lorsque AL=0, la zone est remplie d'espaces.

INT 10H/7 — Défilement vers le bas

AH 7
CH,CL ligne, colonne coin haut gauche
DH,DL ligne, colonne coin bas droit
BH attribut lignes insérées
AL nombre de lignes (0 pour effacer)

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.
Lorsque AL=0, la zone est remplie d'espaces.

INT 10H/8 — Lecture caractère et attribut

AH 8
BH page de texte

AL code caractère
AH attribut (en mode texte)

BX, CX, DX, CS, SS, DS ne sont pas modifiés.
EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/9

Ecriture caractère et attribut
AH 9 AL code caractère BL attribut (si bit 7 à 1 en mode graphique il se produit un XOR avec le contenu de l'écran) BH numéro de page de texte CX nombre de caractères à écrire
BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/0AH

Ecriture caractère
AH 0AH AL code caractère BH page de texte EGA/VGA : BL = couleur du caractère en mode graphique CX nombre de caractères à écrire (répétition)
(En EGA/VGA, si bit 7 de BL est à 1, il se produit, en mode graphique, un XOR avec le contenu de l'écran) BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/0BH

Définition palette/couleur cadre et fond
AH 0BH BH 0 pour définir la couleur du cadre et du fond (en mode texte le cadre seulement) (à n'utiliser qu'en définition 320x200 ou 640x200) 1 pour définir la palette (à n'utiliser qu'en définition 320x200 ou 640x200) BL si BH=0 : couleur à utiliser (bit 0 à 3 + bit 4 pour la brillance) si BH=1 : numéro de palette
BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/0CH

Ecriture pixel
AH 0CH AL couleur CX,DX colonne,ligne EGA/VGA : BH = page écran
BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/0DH

Lecture pixel
AH 0DH CX,DX colonne,ligne EGA/VGA : BH = page écran
AL couleur
BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés.

INT 10H/0EH

Ecriture en mode télétype
AH 0EH AL code du caractère BL couleur du caractère en mode graphique
BX, CX, DX, CS, SS, DS ne sont pas modifiés. EGA/VGA : BX, CX, DX, SI, DI, BP, CS, SS, DS non modifiés. Cette fonction d'écriture comprend le test et l'exécution de la sonnerie.

INT 10H/0FH

Obtention état vidéo
AH 0FH
AL mode vidéo courant AH nombre de colonnes BH numéro de page active affichée
BX, CX, DX, CS, SS, DS ne sont pas modifiés.

INT 10H/10H

Contrôle de couleur et palette (EGA/MCGA/VGA)
AH 10H AL = 0 fixe le contenu d'un registre BH = registre à adresser (de 0 à 15) BL = contenu AL = 1 fixe la couleur du cadre de l 'écran BH = couleur (rangée dans le registre de palette 16) AL = 2 fixe registre de contrôle des attributs ES:DX pointe sur table des couleurs (17 registres de palette) AL = 3 bascule d'intensité/clignotement en alphanumérique BL = 0 si couleur fond intense BL = 1 si clignotement AL = 7 lecture registre de palette (VGA) entrée : BL = numéro de registre de palette sortie : BH = contenu du registre de palette AL = 8 lecture registre de "bordure" (*OverScan*) (VGA) sortie : BH = contenu du registre de "bordure" AL = 9 lecture de tous registres palettes et bordure (VGA) sortie : ES:DX pointe sur le tampon résultat (17 octets)

AL = 10H écriture reg. couleur (convertisseur NA) (VGA)
BX = numéro de registre de couleur (0 à 255)
CH = vert
CL = Bleu
DH = rouge
AL = 12H écriture bloc de reg. couleur (convertisseur NA) (VGA)
BX = numéro de registre de couleur (0 à 255)
CX = nombre de registres à définir
ES:DX = adresse du tampon de définition (CX×3 octets)
AL = 13H sélect. mode couleur (convertisseur NA) (VGA)
BL = 0 : bit 0 de BH recopié dans bit 7 du reg de contrôle de mode (sélection en 4 ou 16 groupes de 64 ou 16 couleurs)
BL = 1 : BH recopié dans registre de sélection de couleur (sélection d'un groupe de couleurs)
AL = 15H lecture d'un reg. couleur (convertisseur NA)
entrée : BX = numéro de registre de couleur (0 à 255)
sortie : CH = vert
sortie : CL = Bleu
sortie : DH = rouge
AL = 17H lecture d'un bloc de reg. couleur (convertisseur NA)
BX = numéro du premier registre de couleur (0 à 255)
CX = nombre de registres à définir
ES:DX = adresse du tampon de définition
AL = 18H défintion du registre de masque (VGA)
BL = valeur du registre de masque
AL = 19H lecture du registre de masque (VGA)
sortie : BL = valeur du registre de masque
AL = 1AH lecture mode de sélection couleur (VGA)
BL = bit 7 du registre de contrôle de mode
BH = valeur du registre de sélection de couleur
AL = 1BH contrôle de sommation de niveaux de gris (VGA)
pour le convertisseur : BX = numéro du registre à convertir
CX = nombre de registres à convertir

BX, CX, DX, SI, DI, BP, CS, SS, DS ne sont pas modifiés.
(voir notes sur l'affichage en fin de chapitre)

INT 10H/11H	Interface du générateur de caractères (EGA/MCGA/VGA)

AH 11H
AL = 0 charge le jeu de caract. défini
BL = numéro de table (0/1)
BH = nombre de lignes par caract.
CX = nombre de caract. dans la table
DX = code ASCII du premier de ces caractères
ES:BP = adresse de la table
AL = 1 charge le jeu de caract. 8x14 (de la ROM vers la RAM)
BL = numéro de table (0/1)
AL = 2 charge le jeu de caract. 8x8 (de la ROM vers la RAM)
BL = numéro de table (0/1)
AL = 3 active un jeu de caractères
BL = numéro de jeu (0/1)
AL = 4 charge des caractères alphanumériques 8x16 (VGA)
BL = numéro de table (0/1)
AL = 10H charge et active le jeu de caract. défini
(idem 0 avec reprog. du contrôleur CRTC)
BL = numéro de table (0/1)
BH = nombre de lignes par caract.
CX = nombre de caract. dans la table
DX = code ASCII du premier de ces caractères
ES:BP = adresse de la table
AL = 11H charge et active le jeu de caract. 8x14
(idem 1 avec reprog. du contrôleur CRTC)
BL = numéro de table (0/1)
AL = 12H charge et active le jeu de caract. 8x8
(idem 2 avec reprog. du contrôleur CRTC)
BL = numéro de table (0/1)
AL = 14H charge et active le jeu de caract. 8x16 (VGA)
BL = numéro de table (0/1)
AL = 20H modifie le vecteur INT 1FH
ES:BP = nouvelle adresse table de caractères (128 à 255)
AL = 21H modifie le vecteur INT 43H
ES:BP = nouvelle adresse de la table de caractères (0 à 255)
CX = nombre de lignes par caractère
si BL = 0 DL indique le nombre de lignes de texte à l'écran
(1 pour 14 lignes, 2 pour 25, 3 pour 43)
AL = 22H fixe l'adresse INT 1FH vers table du BIOS
AL = 23H fixe l'adresse INT 43H vers table du BIOS

AL = 30H lire les informations d'affichage
entrée :
BH = 0 contenu de INT 1FH
BH = 1 contenu de INT 43H
BH = 2 adresse table 8x14
BH = 3 adresse table 8x8
BH = 4 adresse table 8x8 (seconde moitié)
BH = 5 adresse table 9x14 (supplémentaire)
BH = adresse table 8x16 (supplémentaire)
BH = adresse table 9x16 (supplémentaire)
sortie :
CX = hauteur caractères
DL = nombre de colonnes–1
ES:BP = adresse cherchée

BX, CX, DX, SI, DI, BP, CS, SS, DS ne sont pas modifiés sauf : sous-fonction 30H : BX, DH, SI, DI, CS, SS, DS ne sont pas modifiés.

INT 10H/12H — Contrôle des fonctions (EGA/MCGA/VGA)

AH 12H
BL = 10H lire configuration
BL = 20H active recopie d'écran
BL = 30H sélection résolution vert. en alphanum. (EGA/VGA)
0 (200) 1(350) 2 (400 pour VGA)
BL = 31H valid./inhib. chargement par défaut de la palette (VGA)
AL = 0 (valide) / 1 (inhibe)
BL = 32H valid./inhib. adressage de la RAM vidéo (VGA)
AL = 0 (valide) / 1 (inhibe)
BL = 33H valid./inhib. sommation niveaux de gris (VGA)
AL = 0 (valide) / 1 (inhibe)
BL = 34H valid./inhib. émulation curseur (VGA)
AL = 0 (valide) / 1 (inhibe)
BL = 35H valide/inhibe temporairement la sortie écran (VGA)
AL = 0 (valide) / 1 (inhibe)

sortie (lecture de configuration) :
BH = 0 moniteur couleur / haute résolution
BH = 1 moniteur mono.
BL = 0 à 3 pour taille RAM 64 à 256 Ko
CL = 8 moniteur couleur connecté
CL = 9 moniteur EGA /MultiSync.
CL = 0BH moniteur mono.

BX, CX, DX, SI, DI, BP, CS, SS, DS ne sont pas modifiés

INT 10H fonction 13H — Ecriture de chaîne de caractères (PC/AT)

AH 13H
AL 0 attribut dans BL, position du curseur non modifiée
1 attribut dans BL, position du curseur actualisée
2 attribut dans tampon, position du curseur non modifiée
3 attribut dans tampon, position du curseur actualisée
BL octet d'attribut (AL=0 ou 1)
CX nombre de caractères dans la chaîne
DH numéro de ligne
DL numéro de colonne
BH numéro de page
ES:BP : adresse du tampon

Lorsqu'on utilise AL=1 ou 2, le tampon contient contient les caractères avec leur attribut. Si AL=0 ou 1, le tampon ne contient que le code des caractères et ceux-ci sont affichés avec l'attribut défini par BL.
BX, CX, DX, CS, SS, DS ne sont pas modifiés.

INT 10H/1AH — Contrôle de type d'écran (MCGA/VGA)

AH 1AH
AL = 0 lecture du code d'affichage
AL = 1 écriture du code d'affichage

En lecture si AL ≠ 1AH il n'y a pas de BIOS VGA
BL = code carte vidéo active
BH = code carte vidéo non active
FF = carte inconnue
00 = pas de carte
01 = MDA / écran monochrome
02 = CGA / écran CGA
04 = EGA / écran EGA/MultiSync
05 = EGA / écran monochrome
07 = VGA / écran monochrome analogique
08 = MDA / écran couleur analogique

INT 10H/1BH — Lecture d'état du système d'affichage (MCGA/VGA)

AH 1BH
BX 0
ES:DI pointeur sur tampon

AL 1BH

offset	Contenu du tampon

0 à 3	Adresse de la table des informations BIOS (16 octets)
4	Mode vidéo courant
5 à 6	Nombre de colonnes ou points
7 à 8	Taille page écran dans la RAM vidéo
9 à A	Adresse de la page écran active dans la RAM vidéo
B à 1A	Positions curseur dans les pages écran
1B	Ligne de fin du curseur
1C	Ligne de début du curseur
1D	Numéro de la page écran active
1E à 1F	Adresse du contrôleur vidéo
20	
21	
22	Nombre de lignes affichables
23 à 24	Hauteur des caractères en points
25	
26	
27	Nombre de couleurs affichables
29	Nombre de pages écran
2A	
2B	
2C	
2D	
2E à 2F	Réservé
31	Taille de la RAM vidéo
32 à 3F	Réservé

INT 10H/1CH	Sauvegarde et restauration état affichage (VGA)
	AH 1CH AL = 0 donne la taille du tampon d'état AL = 1 sauvegarde de l'état d'affichage AL = 2 restaure l'état d'affichage

IV.1.2. Les fonctions "Super VGA"

Ces fonctions sont appelées avec AH=4FH et un numéro de fonction dans AL.

INT 10H fonction 4F00H	Lecture état (Super VGA)
	AX 4F00H ES:DI : adresse d'un tampon de 256 octets

INT 10H fonction 4F01H	Lecture mode et informations sur le mode (Super VGA)
	AX 4F01H ES:DI : adresse d'un tampon de 256 octets
	CX numéro du mode Informations permettant de traiter des caractéristiques non standard de l'affichage.

INT 10H fonction 4F02H fonction 4F03H	Fixer mode (Super VGA) (2) Lire mode (Super VGA) (3)
	AX 4F02H ou 4F03H BX mode (sous-fonction 2)
	BX mode (sous-fonction 3)

INT 10H fonction 4F04H	Sauvegarde état vidéo(Super VGA)
	AX 4F04H
	Le contenu de tous les registres du contrôleur ainsi que des informations BIOS sont sauvegardées.

En mode 1024×768 l'espace mémoire nécessaire avec une architecture de type VGA est supérieur à 64K. La technique utilisée ressemble à celle qui est mise en œuvre dans la mémoire paginée :

– en mode normal (*Single Window Bank Switching*), le banc 0 permet d'accéder au premier segment d'affichage, le banc 1 au segment suivant, etc. En mode 1024×768, les 512 premières lignes sont accessibles dans le banc 0 et le banc 1 correspond aux 256 lignes suivantes.

– en mode "à fenêtres recouvrantes" (*Overlapping Windows*) on met un banc en *lecture seule*, et l'autre en mode *écriture seule*, pour pouvoir accéder "simultanément" aux deux bancs à travers les mêmes adresses mémoire.

– en mode "à fenêtres non recouvrantes" (*Dual Nonoverlapping Windows*), dans le mode 1024×768, on peut disposer de quatre bancs et les deux fenêtres d'affichage occupant au total trois granules de 32K peuvent être réparties à la demande sur les quatre bancs de 32K.

Le mode super-VGA laisse une certaine liberté aux fabricants de cartes quant au nombre et à la taille des bancs de mémoire sur la carte vidéo.

INT 10H fonction 4FH05	Contrôle des *bancs* vidéo (Super VGA)

	AX 4F05H BH : 0 définition du banc BL : numéro de fenêtre (0 pour A, 1 pour B) DX : numéro du banc BH : 1 lecture du banc

INT 10H fonction 4F06H	Définition largeur d'écran (Super VGA)
	AX 4F06H

INT 10H fonction 4F07H	Définition et lecture adresses d'affichage (Super VGA) (Contrôle d'affichage multi-écran)
	AX 4F07H

IV.1.3. Les fonctions VESA

INT 10H fonction 4F00H	Lire les caractéristiques de la carte (VESA)
	AX 4F00H ES:DI adresse d'un tampon de 256 octets
	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

Le tampon contient les informations suivantes :

offset	contenu
0	marqueur : chaîne "VESA"
4	version, numéro primaire
5	version, numéro secondaire
6	pointeur (4 octets) sur une chaîne ASCIIZ contenant le nom du fabricant de la carte.
0AH	4 octets à 0
OEH	pointeur (4 octets) vers une liste de *mots* codant les modes vidéo soutenus par la carte (le dernier est 0FFFFH).

INT 10H fonction 4F01H	Lire les caractéristiques d'affichage (VESA)

	AX 4F01H CX mode vidéo ES:DI adresse d'un tampon de 29 octets
	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

Le tampon contient les informations suivantes :

offset	contenu
0	mot d'état : bit 0 : 1 si compatibilité carte-moniteur bit 1 : 1 si les informations optionnelles sont bien disponibles bot 2 : 1 support des modes texte du BIOS bit 3 : 1 couleur bit 4 : 1 mode graphique
2	octet d'état première fenêtre d'accès : bit 0 : 1 fenêtre disponible bit 1 : 1 écriture en RAM vidéo activée bit 2 : 1 écriture en RAM vidéo activée
3	octet d'état deuxième fenêtre d'accès : bit 0 : 1 fenêtre disponible bit 1 : 1 écriture en RAM vidéo activée bit 2 : 1 écriture en RAM vidéo activée
4	Granularité pour les déplacements en Ko
6	Taille des fenêtres d'accès en Ko
8	Adresse de segment de la première fenêtre
0AH	Adresse de segment de la seconde fenêtre
0CH	Pointeur
10H	Nombre d'octets pour une ligne de pixels
12H	Nombre de points en X par caractère
14H	Nombre de points en Y par caractère
16H	Largeur de la matrice de points en X par caractère
17H	Hauteur de la matrice de points en X par caractère
18H	Nombre de plans de bits
19H	Nombre de bits / pixel
1AH	Nombre de blocs de mémoire
1BH	Organisation de la mémoire 0 texte, 1 CGA, 2 Hercules, 3 EGA/VGA 16 couleurs, 4 format compacté 2 à 4 bits/pixels, 5 EGA/VGA 256 couleurs
1CH	Taille des blocs de mémoire en Ko

INT 10H fonction 4F02H	Fixer le mode vidéo (VESA)
	AX 4F02H BX mode vidéo (si bit 15=1: pas d'effacement)
	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonction 4F03H	Lire le mode vidéo (VESA)
	AX 4F03H
	BX mode vidéo AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonct.4F04H/0	Lecture/écriture de l'état de la carte (VESA) (Taille tampon nécessaire aux fonctions suivantes)
	AX 4F04H DL 0 CX bit 0 état de la partie vidéo bit 1 zone de données BIOS bit 2 contenu de la table des convertisseurs (DAC) bit 3 état VESA
	BX nombre de blocs de 64 octets nécessaires pour le tampon AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonct.4F04H/1	Ecriture de l'état de la carte (VESA)
	AX 4F04H DL 1 CX même définition que précédemment ES:BX adresse tampon
	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonct.4F04H/2	Lecture de l'état de la carte (VESA)
	AX 4F04H DL 2 CX même définition que précédemment ES:BX adresse tampon

	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonct.4F05H/0	Accès à la RAM vidéo (VESA) (fixer fenêtre d'accès sur la RAM vidéo)
	AX 4F05H BH 0 BL fenêtre d'accès (0/1) DX adresse de départ
	AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

INT 10H fonct.4F05H/1	Accès à la RAM vidéo (VESA) (lire numéro de fenêtre d'accès sur la RAM vidéo)
	AX 4F05H BH 1
	DX adresse de départ AX 004FH si carte VESA et résultat OK AX 014FH si carte VESA et résultat pas OK
	Les registres ne sont pas modifiés.

IV.1.4. Exemples

– Fonction de défilement vers le haut, appelée à partir d'un langage évolué par :

```
CALL DEFILH(ligneH,colonneG,ligneB,colonneB,nombreL)
```

Les paramètres correspondent respectivement aux numéros de ligne haute, de colonne gauche, de ligne basse, de colonne droite et au nombre de lignes de défilement. On suppose que les adresses (seuls les *offsets* sont passés dans la pile; cela correspond au modèle dit *medium*) des arguments sont passées dans la pile dans l'ordre où ceux-ci apparaissent dans l'appel de la procédure. Toutes les variables sont entières.

```
DEFILH procfar
  push    bp
  mov     bp,sp
  add     bp,0Eh
  push    ax
  push    bx
  push    cx
  push    dx
  pushf
  mov     bx,[bp]        ; on lit l'adresse du numéro de ligne haute
  mov     ch,[bx]        ; ch := numéro de ligne haute
```

```
    mov     bx,[bp-2]     ; on lit l'adresse du numéro de colonne gauche
    mov     cl,[bx]       ; ch := numéro de colonne gauche
    mov     bx,[bp-4]     ; on lit l'adresse du numéro de ligne basse
    mov     dh,[bx]       ; ch := numéro de ligne basse
    mov     bx,[bp-6]     ; on lit l'adresse du numéro de colonne droite
    mov     dl,[bx]       ; ch := numéro de colonne droite
    mov     bx,[bp-8]     ; on lit l'adresse du nombre de lignes
    mov     al,[bx]       ; ch := nombre de lignes
    mov     bh,1          ; on force l'attribut des lignes nouvelles à 1
    mov     ah,6          ; code de la fonction
    int     10H
    popf
    pop     dx
    pop     cx
    pop     bx
    pop     ax
    pop     bp
    retf    8
DEFILH      endp
```

– Programme de lecture de la configuration contrôleur écran/écran (MS/BASIC) :

```
DEFINT I-N
TYPE RegTypeX
     ax    AS INTEGER
     bx    AS INTEGER
     cx    AS INTEGER
     dx    AS INTEGER
     bp    AS INTEGER
     si    AS INTEGER
     di    AS INTEGER
     flags AS INTEGER
     ds    AS INTEGER
     es    AS INTEGER
END TYPE
DIM RegIn AS RegTypeX, RegOut AS RegTypeX
CLS
intNum = 16: RegIn.ax = &H1200: RegIn.bx = &H10
CALL interruptX(intNum, RegIn, RegOut)
Mon1 = RegOut.bx AND 255
Mon2 = RegOut.bx \ 256
Mon3 = RegOut.cx
PRINT "-------- contrôle BIOS standard ---------"
PRINT "Taille mémoire carte :"; 64 * (1 + Mon1); "K"
SELECT CASE Mon2
    CASE 0
```



```
        PRINT "Moniteur couleur/haute résolution"
    CASE 1
        PRINT "Moniteur N/B"
    CASE ELSE
        PRINT "?"
END SELECT
SELECT CASE Mon3
    CASE 8
        PRINT "Moniteur couleur connecté"
    CASE 9
        PRINT "Moniteur EGA/MultiSync"
    CASE 11
        PRINT "Moniteur Monochrome"
    CASE ELSE
        PRINT "?"
END SELECT
PRINT "-------- contrôle MCGA/VGA ---------"
'* contrôle type (MCGA/VGA)
RegIn.ax = &H1A00
CALL interruptX(intNum, RegIn, RegOut)
IF ((RegOut.ax AND 255) = &H1A) THEN
    iCodeCA = RegOut.bx AND 255
    PRINT "Code carte activée : "; iCodeCA
    SELECT CASE iCodeCA
        CASE 255
            PRINT "    Type carte inconnu"
        CASE 0
            PRINT "    Pas de carte"
        CASE 1
            PRINT "    MDA/Ecran Monochrome"
        CASE 2
            PRINT "    CGA/Ecran CGA"
        CASE 4
            PRINT "    EGA/Ecran EGA/MultiSync"
        CASE 5
            PRINT "    EGA/Ecran Monochrome"
        CASE 7
            PRINT "    VGA/Ecran Monochrome analogique"
        CASE 8
            PRINT "    VGA/Ecran couleur analogique"
        CASE ELSE
    END SELECT
    iCodeCA = RegOut.bx \ 256
    PRINT "Code carte non activée : "; iCodeCA
    SELECT CASE iCodeCA
        CASE 255
```

```
            PRINT "     Type carte inconnu"
        CASE 0
            PRINT "     Pas de carte"
        CASE 1
            PRINT "     MDA/Ecran Monochrome"
        CASE 2
            PRINT "     CGA/Ecran CGA"
        CASE 4
            PRINT "     EGA/Ecran EGA/MultiSync"
        CASE 5
            PRINT "     EGA/Ecran Monochrome"
        CASE 7
            PRINT "     VGA/Ecran Monochrome analogique"
        CASE 8
            PRINT "     VGA/Ecran couleur analogique"
        CASE ELSE
    END SELECT
ELSE
    PRINT "Il n'y a pas de BIOS VGA"
END IF
Mon1 = RegOut.bx AND 255
Mon2 = RegOut.bx \ 256
Mon3 = RegOut.cx
...
```

IV.2. Notes sur l'affichage

IV.2.1. Attributs et Palettes

En dehors de l'affichage MDA qui est un mode purement *texte*, les types d'affichage des PC disposent de deux modes de fonctionnement : le mode *texte* et le mode *graphique*.

– MDA ou *Hercules* texte : dans ce mode texte, chaque caractère est codé sur deux octets :

– son code ASCII,
– un octet d'attribut définissant l'apparance du caractère :

7	6	5	4	3	2	1	0
(Cl)	fond (F)			(I)	caractère (C)		

Le bit 7 (Cl) est l'*indicateur de clignotement*, le bit 3 (I) celui de *brillance*. Le champ des bits 0 à 2 (C) caractérise le caractère, et les bits 4 à 6 (F) le fond associé. Le premier champ peut prendre les valeur 0 (noir), 1 (souligné) ou 7 (blanc), tandis que le second peut prendre les valeurs 0 (noir) ou 7 (blanc).

– CGA texte 25x80 ou 25x40 : le codage est similaire au précédent. La différence provient des champs (C) et (F) qui contrôlent la couleur du caractère et du fond correspondant. En fait, le bit de brillance permet d'obtenir 8 couleurs supplémentaires pour le caractère, qui en compte ainsi 16, alors que le fonds n'en compte que 8 (à moins que l'on ne désactive le bit de clignotement - bit 5 du registre de sélection de mode 3D8H - celui-ci étant alors considéré comme un bit supplémentaire de couleur).

– CGA graphique 640x200 : chaque octet correspond à huit pixels. La première ligne est associée aux octets d'*offset* 0 à 79, la seconde aux octets 2000_{16} à $2000_{16} + 79$, la troisième aux octets 80 à 159, etc ... Le texte s'inscrit dans une matrice 25x80.

– CGA graphique 320x200 : chaque octet correspond à quatre pixels. A chaque pixel sont associés deux bits codant quatre couleurs ou quatre niveaux de gris dans la palette courante (la couleur 0 peut être choisie librement, tandis que les trois autres correspondent soit à "Turquoise, Violet, Blanc" soit à "Vert, Rouge, Jaune"). L'organisation mémoire est similaire à celle du mode précédent. Le texte s'inscrit dans une matrice 25x40.

– Hercules graphique 720x348 : chaque octet correspond à huit pixels. La première ligne est associée aux octets d'*offset* 0 à 79, la seconde aux octets 2000_{16} à $2000_{16} + 79$, la troisième aux octets 4000_{16} à $4000_{16} + 79$, la quatrième aux octets 6000_{16} à $6000_{16} + 79$, la cinquième aux octets 80 à 159, la sixième aux octets $2000_{16} + 80$ à $2000_{16} + 159$, etc ...

– EGA texte : le codage des caractères est similaire à celui du mode CGA texte. La seule différence est que les champs (C)+(I) et (F) ne codent pas directement la couleur mais donnent accès à 1 parmi 16 registres de *palette* dont la constitution est la suivante :

5	4	3	2	1	0
R	V	B	r	v	b

Chaque bit code une couleur, soit brillante (RVB) soit normale (rvb). On dispose ainsi de 16 couleurs parmi 64.

– VGA texte : la différence avec le mode EGA est que le contenu du registre de palette ne donne plus la couleur mais un index dans un groupe de 256 registres 18 bits (*Digital Analog Converter Color Table* en entrée du convertisseur Digital/Analogique). En principe ceci permet d'obtenir 64 couleurs parmi 256 K couleurs. Etant donnés les 256 registres de la table des couleurs, on peut commuter rapidement entre quatre jeux de 64 couleurs. Il existe un autre mode de fonctionnement dans lequel on n'utilise que les bits 0 à 3 des registres de palette. On dispose alors de 16 jeux de 16 couleurs parmi 256 K.

– EGA graphique 640x350 : chaque pixel est codé de manière interne sur 4 bits (4 *bits Planar Model*). Chaque bit appartient à un plan différent. Chaque écriture ou lecture d'un octet permet d'accéder à 8 bits dans un ou plusieurs plans :

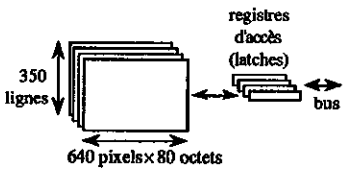

Fig.1 – *Mode EGA graphique*

– En mode VGA graphique, l'organisation de la mémoire vidéo est la même qu'en EGA. Il n'y a qu'un seul mode (mode 13H) dans lequel l'adressage se fait pixel par pixel, la couleur étant codée sur 8 bits. Lorsque le *bit* 7 du registre de contrôle de mode est à 0, on peut schématiser l'affichage selon le principe indiqué figure 2. Les quatre *bits* d'attribut donnent accès à une table de 16 registres 6 *bits*. Ces 6 *bits* fournissent à leur tour un *offset* dans un bloc de 64 registres 18 *bits* de la table des couleurs.

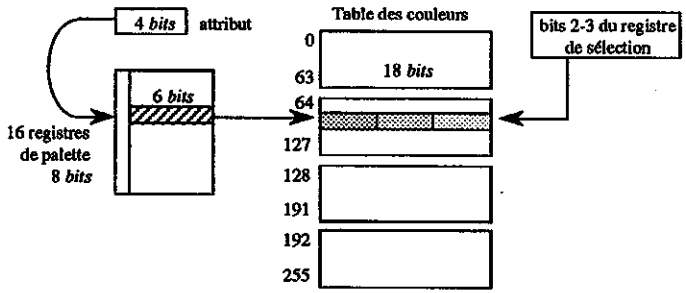

Fig.2 – *Affichage en mode VGA (1)*

La table des couleurs possède de 256 mots de 18 *bits* (6 *bits* pour chacune des couleurs Rouge, Vert et Bleu). Il y a quatre blocs sélectionnés par les *bits* 2 et 3 du registre de sélection de couleur. Si le *bit* 7 du registre de contrôle de mode est à 1, l'affichage est schématisé :

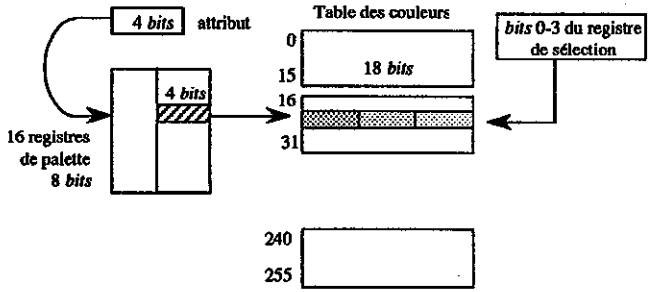

Fig.3 – *Affichage en mode VGA (2)*

Les quatre bits d'attribut donnent toujours accès à la table de palette 16 registres, mais seuls 4 bits seront utilisés. Ces 4 bits fournissent un *offset* dans un bloc de 16 mots de 18 bits. Il y a seize blocs sélectionnés à partir des bits 0, 1, 2 et 3 du registre de sélection de couleur.

IV.2.2. Les contrôleurs

Le contrôle de l'affichage est composé[1] :

– d'un contrôle du signal vidéo *CRTC* (signaux de synchronisation, nombre de lignes de balayage, nombre de points par ligne, gestion du curseur, ...),
– d'un contrôle de séquencement (modes d'affichage texte/graphique, contrôle d'accès aux plans d'affichage, ...
– d'un contrôle de l'accès à la mémoire d'affichage (modes d'écriture et lecture),
– d'un contrôle aux registres d'attributs (couleurs et palettes).

Dans le cas EGA/VGA, les contrôleurs associés ont respectivement pour adresse :

– 3B4-3B5 ou 3D4-3D5 pour le contrôleur *CRTC* (*Cathode Ray Tube Controller*). L'adresse 3Bx ou 3Dx est déterminée par programmation du registre d'adresse 3C2 (*Miscellaneous Output Register*),
– 3C4-3C5 pour le séquenceur,
– 3CE-3CF pour l'accès à la mémoire écran,
– 3C0 pour le contrôle des attributs.

En pratique, la programmation des modes vidéo et de l'accès aux registres de palette se fera en passant par les fonctions du BIOS. Le seul contrôleur auquel on peut avoir besoin d'accéder est le contrôleur d'accès à la mémoire écran lorsqu'on doit manipuler des images en mode graphique. Les fonctions du BIOS, en nombre trop limité, n'offrent, généralement pas une rapidité suffisante.

La plupart de ces contrôleurs possède plusieurs registres, mais seulement deux adresses. L'accès aux registres internes se fait en écrivant, à la première adresse, le numéro du registre auquel on veut accéder, puis, à la seconde, on lit ou écrit la valeur voulue.

Exemple : l'accès à la mémoire écran est fonction des *mode d'écriture et lecture*. Ceux-ci sont généralement égaux à zéro, sauf cas particulier (voir les deux exemples d'utilisation des modes d'écriture 1 et 2). Le registre numéro 5 permet de programmer ce mode :

ØØØØrØww

La programmation du mode *read* 0-*write* 2 (`r=0, w=2`) se fait de la façon suivante :

```
mov   dx,3CEH      ; adresse du registre recevant le numéro de
                   ; registre de contrôle
mov   al,5         ; registre de définition de mode = 5
out   dx,al        ;
inc   dx           ; on passe au registre suivant (3CFH)
```

[1]EGA : Mode d'emploi, Micro-Systèmes, Janvier 1989

```
mov   al,2        ; read 0, write 2
out   dx,al       ;
```

Les modes de lecture et écriture ont les fonctions suivantes[2] :

– *Read* 0 : on accède au contenu des quatre plans. Quatre octets sont recopiés simultanément dans les quatre *latches*. Les deux bits de poids faible du registre *Read Map Select* permet de sélectionner le plan dont l'octet sera ainsi recupéré par le programme de lecture. Pour accéder à huit pixels "dans le même octet", il faudra donc quatre lectures à la même adresse.

– *Read* 1 : sert à comparer la couleur définie par le contenu du registre *Color Compare* avec les couleurs des huit pixels à l'adresse définie par le programme. L'octet lu indique par des 1 lesquels des pixels ont la couleur désignée. Le registre *Color Don't Care* valide ou non les bits impliqués dans la comparaison.

– *Write* 0 : ce mode permet d'accéder aux quatre plans en écriture. Le registre *Bit Mask* indique par des 1 les pixels concernés par l'opération définie par les *bits* 4 et 3 du registre *Function Select*. Les quatre bits de poids faible du registre *Set/Reset* sont combinés respectivement avec les bits de chacun des quatre plans. Le registre *Enable Set/Reset* autorise ou non cette combinaison (le plus souvent ces quatre bits sont à 1).

Lorsque le registre *Enable Set/Reset* est à 0, c'est l'octet émis par le processeur qui est combiné simultanément avec les quatre octets des quatre plans selon la fonction indiquée.

– *Write* 1 : ce mode est destiné aux transferts de mémoire écran à mémoire écran (voir exemple).

– *Write* 2 : ce mode est destiné à manipuler la mémoire écran pixel par pixel. Le registre *Bit Mask* définit les bits qui seront modifiés simultanément dans l'octet adressé. On peut ainsi fixer la couleur de huit pixels en une opération d'écriture (voir exemple). On notera qu'en fait il faut effectuer une opération de lecture-écriture à chaque fois que l'on modifie un pixel pour recharger les *latches*.

Les registres sont les suivants :

	Registre	
0	*Set/Reset Register*	
1	*Enable Set/Reset Register*	
2	*Color Compare Register*	En mode *read*-1 spécifie la couleur que l'on compare aux pixels de l'octet adressé.
3	*Function Select Register*	Les bits 4 et 3 indiquent la fonction à réaliser : – 00 : on remplace, – 01 : ET logique, – 10 : OU logique, – 11 : OU exclusif
4	*Read Map Select Register*	
5	*Mode Register*	Définition des modes de lecture/écriture
6	*Miscellaneous Register*	
7	*Color Don't Care Register*	En mode *read*-1, indique quels pixels de l'octet adressé on exclue de la comparaison.
8	*Bit Mask Register*	Un 1 indique que le pixel correspondant de l'octet adressé sera impliqué dans l'opération d'écriture.

[2] La bible PC, Editions Micro Applications

IV.2.3. Exemples

Sous-programme de décalage vers la gauche d'une zone d'écran graphique EGA/VGA alignée horizontalement sur des limites d'octets. Les arguments d'appels sont les suivants :

– `iCoulF` : couleur de fond pour la partie décalée,
– `ipas` : nombre d'octets de décalage,
– `icol` : colonne gauche (c'est un multiple de huit),
– `indc` : nombre d'octets constituant la ligne,
– `ilmin` : ligne haute,
– `indMax` : nombre de lignes–1.
Le mode d'écriture utilisé est le mode 1.

```
;*---------------- appel ------------------------------*
;*  CALL decaleG(iCoulF,ipas, icol, indc, ilMin, indMax) *
;*-------------------------------------------------------*
        public  decaleG
decaleG proc    far
        jmp     debdecG
FarPtr  dd      ?
indl    dw      ?
indc    dw      ?
ilMin   dw      ?
colonne dw      ?
indice  dw      ?                           ; num. de ligne relatif
TabOr   db      0,1,2,4,8,16,32,64,128
cte     dw      80                          ; 80 colonnes
saveDI  dw      ?
ipas    dw      ?
ohg     dw      ?
interr  db      ?
iCoulF  db      ?

debdecG label   near
        push    bp
        mov     bp, sp
        pusha
        push    es
        pushf
        mov     bx,[bp+6]                   ; nbre de lignes
        mov     ax,[bx]
        mov     word ptr[indl],ax
        mov     bx,[bp+8]                   ; idem pour la ligne haute
        mov     ax,[bx]
        mov     word ptr[ilMin],ax
        mov     bx,[bp+10]                  ; idem pour la colonne max
        mov     ax,[bx]
```

```
        mov     word ptr[indc],ax
        mov     bx,[bp+12]                  ; idem pour la colonne gauche
        mov     ax,[bx]
        mov     word ptr[colonne],ax
        mov     bx,[bp+14]                  ; idem pour pas de décalage
        mov     ax,[bx]
        mov     word ptr[ipas],ax
        mov     bx,[bp+16]                  ; couleur du fond
        mov     al,[bx]
        mov     byte ptr[iCoulF],al

        mov     ax,[indc]                   ; nbre de décalages
        sub     ax,[ipas]                   ; = indc - ipas
        mov     [indc],ax

        mov     dx,3CEH                     ; registre d'adresse
        mov     ax,105H                     ; on met 1 dans reg5
        out     dx,ax                       ; read 0, write 1
        mov     ax,0A000H                   ; segment EGA/VGA
        mov     es,ax                       ; dans ES

        mov     ax,[ilMin]                  ; on initialise les registres
        mul     word ptr[cte]               ; ilMin*80+iCol
        add     ax,[colonne]
        mov     [ohg],ax
        mov     di,ax                       ; DI=offset octet haut gauche
        mov     [saveDI],ax
        mov     si,ax
        add     si,[ipas]                   ; octet suivant
        mov     cx,[indl]
        inc     cx                          ; on répète indl+1 fois

bclDecG :
        push    cx
        mov     cx,word ptr[indc]
        rep     movs es:byte ptr[di],es:[si]  ; décale indc caract.
        add     [saveDI],80                 ; on passe à la ligne
        mov     ax,[saveDI]                 ; suivante
        mov     di,ax
        mov     si,ax
        add     si,[ipas]
        pop     cx
        loop    bclDecG

        ;-------- on efface ce qui reste -------------------------
        ; bit mask register reg8=FF par défaut (tous les bits)
```



```
        ; function select   reg3=00 par défaut (preset)
        ; enable set/reset  reg1=00 par défaut (->F tous plans)
        ; set/reset         reg0=00 par défaut (-> code couleur)
        ; mode write 0      reg5=00
        ;-----------------------------------------------------------
        mov     dx,3CEH                     ; registre d'adresse
        mov     ax,005H                     ; on met 0 dans reg5
        out     dx,ax                       ; mode write 0
        mov     dx,3CEH                     ; registre d'adresse
        mov     ax,0F01H                    ; on met F dans reg1
        out     dx,ax                       ;
        mov     dx,3CEH                     ; registre d'adresse
        xor     al,al                       ; on met iCoulF dans reg0
        mov     ah,[iCoulF]
        out     dx,ax                       ; (marron)

        mov     ax,[ohg]                    ; point haut gauche
        add     ax,[indc]                   ; on ajoute indc
        mov     [saveDI],ax
        mov     di,ax
        mov     cx,[indl]
        inc     cx                          ; on répète indl+1 fois
bclDecG2 :
        push    cx
        mov     cx,word ptr[ipas]
bclDecG3 :
        mov     es:byte ptr[di],ah
        inc     di
        loop    bclDecG3

        add     [saveDI],80                 ; on passe à la ligne
        mov     ax,[saveDI]                 ; suivante
        mov     di,ax
        pop     cx
        loop    bclDecG2

        popf
        pop     es
        popa
        pop     bp
        retf    12
decaleG endp
```

Exemple de sous-programme d'affichage en mode graphique EGA/VGA : un tampon contient les couleurs, sur huit bits, de chacun des pixels du rectangle à afficher. Les arguments d'appels sont

passés par adresse, ici des *adresses courtes*, c'est-à-dire uniquement les *offsets* des variables dans le segment de données courant (*model medium*) :

- `iColG` : colonne gauche,
- `iColD` : colonne droite,
- `iLigH` : ligne haute,
- `iLigB` : ligne basse,
- `SegTamp` : segment du tampon contenant la couleur des pixels,
- `PtrTamp` : *offset* du tampon.

Le code est du code 286. Le mode d'écriture 2 est utilisé le plus souvent pour dessiner un pixel dont la couleur est fournie par le programme. Le pixel allumé est défini par un masque (contenu du registre 8), un bit à 1 indiquant celui qui est concerné.

```
        .286
        .SALL
.MODEL MEDIUM
.CODE
;*-------------- Appel ----------------------------------------
;* CALL PlotTab(iColG,iColD,iLigH,iLigB,SegTamp,PtrTamp)
;*-----------------------------------------------------------------
SegEGACO equ    0A000H
        public  PlotTab
PlotTab proc    far
        jmp     debPlotTab
iColG   dw    ?       ; colonne gauche
iColD   dw    ?       ; colonne droite
iLigH   dw    ?       ; ligne haute
iLigB   dw    ?       ; ligne basse
kBitG   db    ?       ; position 1° pixel dans l'octet de début de ligne
kBitD   db    ?       ; position dernierpixel dans dernier octet ligne
iCGs8   dw    ?       ; offset 1° octet dans segment d'affichage
iCDs8L  dw    ?       ; offset dernier octet (sur la même ligne iLigH)
iCDs8   dw    ?       ; offset dernier octet (sur iLigB)
NbCol   dw    ?       ; nombre de colonnes
NbLig   dw    ?       ; nombre de colonnes
NbOct   dw    ?       ; nombre d'octets complets dans la ligne
;
debPlotTab   label   near
    push    bp
    mov     bp, sp
    pusha
    push    es
    push    ds
    pushf
    mov     bx,SegEGACO
    mov     es,bx
    ;*--------- traitement colonne gauche
```



```
    mov     bx,[bp+16]              ; Colonne gauche
    mov     ax,[bx]
    mov     word ptr cs:[iColG],ax
    mov     bx,ax
    and     ax,7                    ; iColG MOD 8
    mov     byte ptr cs:[kBitG],al
    shr     bx,3                    ; iColG \ 8
    mov     word ptr cs:[iCGs8],bx
    mov     word ptr cs:[NbOct],bx
    ;*--------- traitement colonne droite
    mov     bx,[bp+14]              ; Colonne droite
    mov     ax,[bx]
    mov     word ptr cs:[iColD],ax
    mov     word ptr cs:[NbCol],ax
    mov     bx,ax
    and     ax,7                    ; iColD MOD 8
    mov     byte ptr cs:[kBitD],al
    shr     bx,3                    ; iColD \ 8
    mov     word ptr cs:[iCDs8],bx
    mov     word ptr cs:[iCDs8L],bx
    dec     bx
    sub     bx,word ptr cs:[NbOct]
    jns     CalcNbCol
    xor     bx,bx
CalcNbCol:
    mov     word ptr cs:[NbOct],bx
    ;*--------- nBCol = (iColD - iColG + 1) --- nombre de colonnes
    mov     ax,word ptr cs:[iColG]
    sub     word ptr cs:[NbCol],ax
    inc     word ptr cs:[NbCol]
    ;*--------- traitement ligne haute
    mov     bx,[bp+12]
    mov     ax,[bx]                 ; Ligne haute
    mov     word ptr cs:[iLigH],ax
    mov     cx,80
    mul     cx                      ; calcul offset premier octet
    add     word ptr cs:[iCGs8],ax
    add     word ptr cs:[iCDs8L],ax ; es:[di] sera l'adresse en
    mov     di,word ptr cs:[iCGs8]  ; mémoire écran
    ;*--------- traitement ligne basse
    mov     bx,[bp+10]              ; Ligne basse
    mov     ax,[bx]
    mov     word ptr cs:[iLigB],ax
    mov     word ptr cs:[NbLig],ax
    mul     cx                      ; Calcul offset dernier octet
    add     word ptr cs:[iCDs8],ax
```

```
    ;*--------- NbLig = (iLigB - iLigH + 1) --- nombre de lignes
    mov     ax,word ptr cs:[iLigH]
    sub     word ptr cs:[NbLig],ax
    inc     word ptr cs:[NbLig]
    ;*--------- offset iTab(0)
    mov     bx,[bp+6]
    mov     ax,[bx]
    mov     si,ax
    ;*--------- segment iTab(0)
    mov     bx,[bp+8]
    mov     ax,[bx]
    mov     ds,ax           ; ds:[si] est l'adresse dans la table

;*-------------- on commence le traitement
    mov     dx,3CEH
    mov     ax,205H   ; 5=registre de mode, 2=(écriture 2, lecture 0)
    out     dx,ax     ; on écrit le mode
    mov     ax,300H   ; 3=reg."Function Select Reg." en mode "preset"
    out     dx,ax     ;
    mov     al,8      ; numéro du registre de masquage
    out     dx,al
    inc     dx        ; on accède maintenant au registre de masquage

    mov     ax,word ptr cs:[iCGs8]  ; on regarde si octets gauche et
    cmp     ax,word ptr cs:[iCDs8L] ; droit sont les mêmes
    mov     cx,word ptr cs:[NbLig]
    jne     bcl2PlotTab
;*---------------------------- un seul octet --------------------
    mov     bh,byte ptr cs:[kBitG]
    mov     bl,byte ptr cs:[kBitD]
bcl1PlotTab:
    call    PlotEGA
    add     di,80               ; on passe à la ligne suivante
    loop    bcl1PlotTab
    jmp     finPlotTab

;*---------------------------- plusieurs octets ----------------------
bcl2PlotTab:                        ;*--------------------------------
    mov     bh,byte ptr cs:[kBitG]
    mov     bl,7
    push    di
    call    PlotEGA                 ; premier octet --------
    inc     di

    push    cx
    mov     cx,word ptr cs:[NbOct]  ; octets entiers -------
```



```
    or      cx,cx
    jz      dernOct
bcl3PlotTab:
    mov     bx,7                    ; pixel gauche=0, droit=7
    call    PlotEGA
    inc     di
    loop    bcl3PlotTab
dernOct:
    pop     cx
    xor     bh,bh
    mov     bl,byte ptr cs:[kBitD]  ; dernier octet --------
    call    PlotEGA
    pop     di
    add     di,80           ; on passe à la ligne suivante
    cmp     si,8000H
    jbe     NoCorrection
    sub     si,1000H
    mov     ax,ds
    add     ax,100H
    mov     ds,ax
NoCorrection:
      loop    bcl2PlotTab         ;*--------------------------------

finPlotTab:
    mov     dx,3CEH
    mov     ax,0FF08H    ; masquage validé
    out     dx,ax       ;
    ;*--- puis on revient en mode Read 0, Write 0
    mov     ax,5        ; 5=reg. de mode, 0=(écriture 0, lecture 0)
    out     dx,ax       ; on écrit le mode

    popf
    pop     ds
    pop     es
    popa
    pop     bp
    retf    12
PlotTab endp

;*-----------------------
;* bh position premier pixel dans octet (0 = bit de gauche)
;* bl position dernier pixel dans octet (7 = bit de droite)
;* al sert d'octet de masquage
;* dx est l'adresse du port VGA=3CFH (on a déjà mis 8 en 3CEH
;* pour désigner le registre de masquage)
;*-----------------------
```

```
PlotEGA proc near
    push    cx
    ;*------ masque=2^(7-bh) -> AL
    mov     cl,bh
    inc     cl
    xor     al,al
    stc
    rcr     al,cl

    ; nombre de pixels concernés dans l'octet
    mov     cl,bl
    sub     cl,bh
    inc     cl
    xor     ch,ch       ; pour avoir CX=compteur de boucles

bclePlotEGA:                        ;*--------------------------------
    out     dx,al       ; on écrit le masque
    shr     al,1        ; on passe au pixel suivant
    mov     ah,byte ptr [si]    ; lecture d'un octet dans table
    inc     si
    xchg    ah,byte ptr es:[di]
    loop    bclePlotEGA         ;*--------------------------------

    pop     cx
    ret
PlotEGA endp

PLOT_TEXT ends
    end
```

Exemple de sous-programmes de lecture d'un plan complet d'écran :

```
            .286
            .SALL
.MODEL MEDIUM
.CODE
;
;*------------------------------------------------------------*
;*                    AFFICHAGE D'UN PLAN                     *
;*------------------------------------------------------------*
;* Format d'appel Basic :                                     *
;* CALL PlotPlan(BYVAL noplan,BYVAL nblig,BYVAL i1,BYVAL i2)  *
;*------------------------------------------------------------*
;* Définition des paramètres :                                *
;* noplan = 1,2,4,8 (n° du plan)                              *
;* nblig = nombre de lignes d'affichage (480 ou 350)          *
```



```
;* i1     = VARSEG(iTab(0))                                   *
;* i2     = VARPTR(iTab(0))                                   *
;*------------------------------------------------------------*
        public  PlotPlan
PlotPlan proc   far
        push    bp
        mov     bp,sp
        pusha
        push    es
        push    ds
        push    ss
        pushf
        ;-------- on récupère l'adresse du tableau dans DS:SI
        mov     si,[bp+6]               ; offset iTab(0)
        mov     bx,[bp+8]               ;
        mov     ds,bx                   ; segment iTab(0)
        ;-------- nombre d'octets par ligne
        mov     cx,80
        ;-------- on récupère le nombre de lignes
        mov     ax,[bp+10]              ;
        mul     cx                      ;
        mov     cx,ax                   ; nbre d'octets dans CX

        mov     di,0                    ; adresse plan vidÇo
        mov     ax,0A000H               ;
        mov     es,ax                   ; dans ES:DI

        cld
        mov     dx,3CEH                 ; enable set/reset (n° du plan)
        mov     ax,1003h                ; fonction select 2 (ou logique)
        out     dx,ax
        mov     ax,0f00h                ; set/reset register
        out     dx,ax

        mov     al,1
        mov     ah,[bp+12]
        out     dx,ax
        mov     al,8                    ; mask register
        out     dx,al
        inc     dx
PlotPlanLab1:
        outsb
        mov     al,es:[di]
        mov     es:[di],al
        inc     di
        loop    PlotPlanLab1
```

```
        popf
        pop     ss
        pop     ds
        pop     es
        popa
        pop     bp
        retf    8
PlotPlan endp

;*-----------------------------------------------------------*
;*                    LECTURE D'UN PLAN                      *
;*-----------------------------------------------------------*
;* Format d'appel Basic :                                    *
;* CALL ReadPlan(BYVAL noplan,BYVAL nblig,BYVAL i1,BYVAL i2) *
;*       (passage par valeur des paramètres)                 *
;*-----------------------------------------------------------*
;* Définition des paramètres :                               *
;* noplan = 0,1,2,3 (n° du plan)                             *
;* nblig  = nombre de lignes d'affichage (480 ou 350)        *
;* i1     = VARSEG(iTab(0))                                  *
;* i2     = VARPTR(iTab(0))                                  *
;*-----------------------------------------------------------*
        public  ReadPlan
ReadPlan proc   far
        push    bp
        mov     bp, sp
        pusha
        push    es
        push    ds
        push    ss
        pushf
        ;-------- on récupère l'adresse du tableau dans ES:DI
        mov     di,[bp+6]               ; offset iTab(0)
        mov     bx,[bp+8]               ;
        mov     es,bx                   ; segment iTab(0)
        ;-------- nombre d'octets par ligne
        mov     cx,80
        ;-------- on récupère le nombre de lignes
        mov     ax,[bp+10]              ;
        mul     cx                      ;
        mov     cx,ax                   ; nbre d'octets dans CX

        mov     si,0                    ; adresse plan vidéo
        mov     ax,0A000H               ;
        mov     ds,ax                   ; dans ES:DI
```



```
        cld
        mov     dx,3CEH                 ;
        mov     al,4
        mov     ah,[bp+12]              ; n° de plan à sauvegarder
        out     dx,ax
        rep     movsb                   ; transfert

        popf
        pop     ss
        pop     ds
        pop     es
        popa
        pop     bp
        retf    8
ReadPlan endp

PLOTPLAN_TEXT ends
        end
```

CHAPITRE V

V. Les fonctions du BIOS

Les fonctions décrites ici ne comprennent pas les interruptions 10H (chapitre sur l'affichage) et 13H (chapitre sur les disques).

V.1. Lecture du vecteur d'équipement (INT 11H)

INT 11H	Lecture vecteur d'équipement (PC/XT)
	AX vecteur d'équipement (0040:0010) *bit* – 15-14 nombre d'imprimantes – 12 présence de l'adaptateur de jeux – 11-10-9 nombre de liaisons série – 8 présence de canal DMA – 7-6 nombre de lecteurs de disques souples (si bit 0=1) – 5-4 type vidéo au démarrage – 3-2 taille mémoire sur carte mère (par blocs de 16K) – 0 existence de lecteur de disquette
	Seul AX est modifié.

INT 11H	Lecture vecteur d'équipement (AT)
	AX vecteur d'équipement (0040:0010) – 15-14 nombre d'imprimantes – 11-10-9 nombre de liaisons série – 7-6 nombre de lecteurs de disques souples – 5-4 type vidéo au démarrage) – 1 (à 1 si coprocesseur) – 0 existence de lecteur de disquette
	Seul AX est modifié.

V.2. Lecture de la taille mémoire (INT 12H)

INT 12H	Obtention taille mémoire
	AX nombre de Ko

V.3. Fonctions de contrôle des ports série (INT 14H)

Octet d'état du canal :

bit	Signification
0	Caractère reçu
1	Caractère écrit dans le registre de données
2	Erreur de parité
3	Procole non conservé
4	Connexion interrompue
5	Registre de données vide
6	Registre à décalage vide
7	*Time-out*

Octet d'état du modem :

bit	Signification
0	Modem prêt à émettre (*)
1	Modem activé (*)
2	Détection front arrière de sonnerie (*)
3	Liaison modem établie (*)
4	Modem prêt à émettre
5	Modem activé
6	Détection front arrière de sonnerie
7	Liaison modem établie

(*) 1 indique qu'il y a eu un changement d'état

INT 14H/0	Initialisation du port de communication
	AH 0 AL paramètres de transmission – *bits* 1-0 : 2 = 7 *bits*, 3 = 8 *bits* – *bits* 2 : 0 = 1 *bit* de STOP, 1 = 2 *bits* de STOP – *bits* 5-4 : 0 pas de parité, 1 parité impaire, 2 parité paire – *bits* 7-6-5 : de 0 à 7 : 110, 150, 300, 600, 1200, 2400, 4800, 9600
	AH état de la ligne AL état du modem

INT 14H/1	Envoi de caractère
	AH 1 AL caractère
	AH – *bit* 7 à 0 caractère transmis – *bit* 7 à 1 caractère non transmis Les autres *bits* contiennent l'état du canal

INT 14H/2	Réception de caractère
	AH 2
	AL caractère reçu AH 0 pas d'erreur sinon état du canal

INT 14H/3	Etat chaîne de communication
	AH 3
	AL état modem AH état canal

V.4. L'interruption INT 15H

L'interruption 15H n'est pas utilisée de la même façon sur tous les modèles *XT*, *AT* ou *PS/2*. Seules quelques fonctions sont décrites ci-après. Certaines autres servent au système d'exploitation et non au BIOS. La touche SYS (PC-AT), en particulier, appelle l'interruption 15H (appelée aussi *interruption cassette*) en transmettant AH=85H. Si AL vaut 0 c'est que la touche est activée. Si AL vaut 1, c'est qu'elle a été relâchée.

INT 15H	Interruption cassette ou touche SYS
	La touche SYS peut facilement être utilisée pour des applications particulières en déroutant celle-ci.

Autres fonctions de INT 15H (PC-AT) :

INT 15H/83H	Horloge temps réel
	AH 83H CX:DX nombre de μs du délai ES:BX drapeau
	Le drapeau ne peut être testé qu'après qu'après fin du délai.

INT 15H/86H	Fonction d'attente
	AH 86H CX:DX nombre de μs du délai
	Le programme ne reprend la main qu'après fin du délai.

INT 15H/87H	Transfert de mémoire (passage au mode protégé du 80286)
	AH 87H CX nombre de mots à déplacer ES:SI adresse de la GDT

	AH code état – 0 pas d'erreur – 1 erreur de parité sur la mémoire – 2 GDT incorrecte – 3 le mode protégé n'a pu être initialisé

INT 15H/88H	Lecture de la taille mémoire étendue
	AH 88H
	Taille mémoire étendue en Ko

INT 15H/89H	Passage en mode protégé
	AH 89H

INT 15H/0C0H	Acquisition adresse des paramètres de configuration
	AH 0C0H
	Table des paramètres (octets 6 à 9 : réservés) : – octets 0-1 : longueur de la table qui suit (8) – octet 2 : identification du modèle – octet 3 : type – octet 4 : version BIOS – octet 5 : mot de configuration

V.5. Fonctions de contrôle clavier (INT 16H)

INT 16H/0	Lecture de caractère
	AH 0
	AX code ASCII / code géographique
	L'en-tête du tampon clavier avance dans la file d'attente du clavier.

INT 16H/1	Etat du tampon clavier
	AH 1
	Z 0 AX = *scan code* / code ASCII 1 aucun code disponible
	L'en-tête du clavier n'est pas modifiée

INT 16H/2	Drapeaux clavier
	AH 2
	AL drapeaux clavier – *bit* 0 SHIFT droit enfoncé – *bit* 1 SHIFT gauche enfoncé – *bit* 2 CONTROL enfoncé – *bit* 3 ALT enfoncé – *bit* 4 état du verrouillage du défilement – *bit* 5 état du verrouillage des touches num. – *bit* 6 état du verrouillage des majuscules – *bit* 7 état touche INSERTION

INT 16H/3	Réglage pour la répétition automatique des touches
	AX 0305H BH délai jusqu'à répétition de 1/4*s* à 1*s* BL vitesse de répétition
	Dépend du BIOS

INT 16H/5	Met un code dans le tampon clavier
	AH 05H CX *scan code* / code ASCII
	AL 0 opération effectuée 1 tampon saturé
	Dépend du BIOS

INT 16H/10H	Test de clavier étendu
	AH 10H
	AX *scan code* / code ASCII
	Comme pour la fonction 0 mais pour les claviers étendus du type AT, Dépend du BIOS

INT 16H/11H	Test de clavier étendu
	AH 11H
	Z 0 AX = *scan code* / code ASCII (Si AL=0, AH=code ASCII pour clavier étendu), 1 aucun code disponible
	Dépend du BIOS

V.6. Fonctions de contrôle port parallèle (INT 17H)

INT 17H/0	Impression de caractère
	AH 0 AL code du caractère DX numéro d'imprimante
	AH état

INT 17H/1	Initialisation du port imprimante
	AH 1 DX numéro d'imprimante
	AH état

INT 17H/2	Etat imprimante
	AH 2 DX numéro d'imprimante

	AH état – *bit* 0 fin de temporisation (*time out*) – *bit* 3 erreur d'E/S – *bit* 4 sélection – *bit* 5 fin de papier – *bit* 6 ACK – *bit* 7 imprimante occupée (0 si BUSY)

V.7. Autres fonctions

V.7.1. L'appel au moniteur (INT 18H)

INT 18H	Appel moniteur

V.7.2. Le pré-chargement (INT 19H)

L'appel à cette interruption est décrit dans le chapitre II et se produit notamment lorsqu'on appuie simultanément sur les touches <CTRL–ALT–DEL> pour provoquer un redémarrage de la machine.

INT 19H	Pré-amorce

V.7.3. Gestion de l'heure et de la date (INT 1AH)

INT 1AH/0	Lecture de l'heure
	AH 0
	CX:DX temps écoulé depuis 0h en unités de 55 *ms*. AL nombre de fois où le compteur est passé à 0 depuis le dernier appel à cette fonction.
	BX, SI, DI, BP, CS, DS, SS, ES non modifiés

INT 1AH/1	Mise à l'heure
	AH 1 CX:DX temps écoulé depuis 0h en unités de 55 *ms*
	AX, BX, CX, DX, SI, DI, BP, CS, DS, SS, ES non modifiés.

INT 1AH/2	Lecture de l'horloge (AT)
	AH 2
	CY=0 CH = heure CL = minutes DH = secondes CY=1 défaut de pile
	Données au format BCD BX, SI, DI, BP, CS, DS, SS, ES non modifiés

INT 1AH/3	Mise à jour de l'horloge (AT)
	AH 3 CH heure CL minutes DH secondes DL 0 (heure d'hiver) ou 1 (heure d'été)
	Données au format BCD BX, SI, DI, BP, CS, DS, SS, ES non modifiés

INT 1AH/4	Lecture de la date sur l'horloge (AT)
	AH 4
	CY=0 CH = siècle (19 ou 20) CL = année DH = mois DL = jour CY=1 défaut de pile

INT 1AH/5	Mise à jour de la date sur l'horloge (AT)
	AH 5 CH siècle (19 ou 20) CL année DH mois DL jour
	Données au format BCD BX, CX, SI, DI, BP, CS, DS, SS, ES non modifiés

INT 1AH/6	Fixer l'heure d'alarme (AT)
	AH 6 CH heure CL minutes DH secondes
	CY 1 : problème de piles ou une alarme a déjà été programmée (il faut la désactiver par la fonction 7)
	Les données sont au format BCD. Lorsque l'heure d'alarme est atteinte, un appel à l'interruption 70H est réalisé. Ensuite est effectué un INT 4AH qui est à la libre disposition des utilisateurs. BX, CX, SI, DI, BP, CS, DS, SS, ES non modifiés

INT 1AH/7	Désactiver une alarme (AT)
	AH 7
	BX, CX, SI, DI, BP, CS, DS, SS, ES non modifiés

V.7.4. La touche <CTRL BREAK> (INT 1BH)

Cette touche, lorsqu'elle est utilisée, appelle l'interruption 1BH à disposition des utilisateurs.

INT 1BH	Interruption <CTRL BREAK>

V.7.5. La fonction INT 1CH

A chaque *top* d'horloge (18,2 *ticks/s* = 65536/3600), l'interruption 1CH est appelée. Cette interruption est laissée à la disposition des utilisateurs.

INT 1CH	Traitement horloge utilisateur

V.7.6. Contrôle de disquette (INT 40H)

L'interruption 40H, utilisée pour le contrôle des disquettes, est destinée à remplacer INT 13H lorsque cette dernière est chargée de gérer les disques durs.

INT 40H	voir INT 13H

CHAPITRE VI

VI. Les fonctions du DOS

Les fonctions compatibles XENIX sont indiquées par (*). Les fonctions non documentées dans le DOS sont décrites dans un cadre grisé. Les fonctions qui remplacent les appels documentés sont notées en caractères gras.

VI.1. Les fonctions de bas niveau

INT 20H	1	Fin d'exécution
	CS	Segment PSP
	Cet appel est utilisé dans les programmes .COM développés dans des versions antérieures à 2.0 de MSDOS (utiliser plutôt fonction 4CH de INT 21H). Tous les *Handles* sont fermés. Tous les tampons disque sont sauvegardés sur disque. Les adresses rangées aux *offset* 0AH (*Terminate Address*), 0EH (traitement du <CTRL-C>) et 12H (*Critical Error*) sont récupérées par le programme *père*.	

Les interruptions 25H et 26H permettent d'accéder aux secteurs d'un disque par un numéro logique. Le secteur de *boot* possède le numéro 0. A partir de la version 4.0 du DOS il y a un changement si les partitions sont de taille supérieure à 32 Mo. Dans ce cas CX doit contenir FFFFH et DS:BX l'adresse d'un bloc de données contenant :

offset	contenu
0-3	numéro du premier secteur
4-5	nombre de secteurs
6-9	pointeur sur le tampon de transfert

INT 25H	1	Lecture absolue disque
	AL	numéro d'unité disque (0 pour A, 1 pour B, 2 pour C, etc ...)
	DS:BX	adresse de transfert disque (DTA)
	CX	nombre de secteurs
	DX	numéro du premier secteur relatif (le premier est 0)

	AL	code d'erreur si CF=1
	flag CF	indicateur d'erreur
	Tous les registres exceptés ceux de segments sont modifiés. !!! Le système sauvegarde les drapeaux sur la pile au moment de l'appel de cette fonction. Il est donc indispensable de faire un POPF après appel à cette fonction.	

INT 26H	1	Ecriture absolue disque
	AL	numéro d'unité disque
	DS:BX	adresse de transfert disque (DTA)
	CX	nombre de secteurs
	DX	numéro du premier secteur relatif (le premier est 0)
	AL	code d'erreur
	flag CF	indicateur d'erreur
	Tous les registres sauf ceux de segments sont modifiés. !!! le système sauvegarde les drapeaux sur la pile au moment de l'appel de cette fonction. Il est donc indispensable de faire un POPF après appel à cette fonction.	

INT 27H	1	Fin d'exécution en restant résident
	CS:DX	Adresse du premier octet suivant le code du programme appelant.
	Cet appel est utilisé dans les programmes .COM développés dans des versions antérieures à 2.0 de MSDOS (voir fonction 31H de INT 21H).	

INT 28H	Inoccupation DOS (→ **fonction 1680H de INT 2FH**)
	Keyboard Busy Loop Utilisé lorsqu'on est sous contrôle du clavier pour savoir si l'on peut faire appel au DOS (sert essentiellement lors de l'écriture de programme résidents faisant appel au DOS).

INT 2EH		
	DS:SI	adresse d'une chaîne de commande
	Execute Command. La chaîne de commande est transmise à la partie résidente de COMMAND.COM. Tous les registres sont modifiés.	

L'interruption 2FH dite *interruption multiplex* permet aux divers programmes résidents de fournir leurs services aux autres programmes (*TSR* et *device drivers*). Comme pour les autres interruptions, le registre AH doit contenir l'identificateur de la fonction demandée. Un certain nombre de numéros sont réservés à des utilitaires du DOS :

AH (hexa)	Utilitaire
1	PRINT.EXE
6	ASSIGN.COM
10	SHARE.EXE
11	(réseau)
14	NLSFUNC.EXE
1A	ANSI.SYS
43	HIMEM.SYS
48	DOSKEY.COM
4B	(gestion des tâches)
AD	KEYB.COM
AE	APPEND.EXE
B0	GRAFTABL.COM
B7	APPEND.EXE

Les adresses indiquées aux "adresses" INT 22H, INT 23H, INT 24H fournissent :

– INT 22H : donne l'adresse de fin de traitement. Cette adresse est copiée à l'*offset* 0AH du PSP lorsque celui-ci est créé.

– INT 23H : donne l'adresse de traitement du <CTRL-C>. Cette adresse est copiée à l'*offset* 0EH du PSP. Cette adresse de traitement est utilisée si l'on tape sur <CTRL-C> lors d'une entrée de caractères sur le clavier, ou pendant un affichage.

– INT 24H : donne l'adresse de traitement d'erreur *fatale*. Celle-ci est copiée à l'*offset* 12H du PSP. Cette interruption n'est pas utilisée par INT 25H et INT 26H. COMMAND.COM possède en effet son propre traitement (envoi du message : *Retry, Abort, Ignore ?*).

Lorsqu'une erreur se produit, les registres AX et DI contiennent une information concernant la cause de l'erreur. Si le *bit* 7 de AH est à 1 l'erreur est dûe à une image erronée de la FAT ou une erreur sur un *character device*. Le bloc de données (*Device Header*) peut, dans ce dernier cas, être examiné à l'adresse BP:SI (le bit de poids fort de l'octet d'attribut indique de quel type d'unité il s'agit. On peut en déduire s'il y a erreur sur une FAT ou à cause d'une unité d'E/S de type "caractère").

Code erreur	Signification
0	Tentative d'écriture sur disque protégé en écriture.
1	Unité inconnue.
2	Lecteur pas prêt.
3	Commande inconnue.
4	Erreur sur les données.
5	Longueur de la structure de données de requête incorrecte.
6	Erreur de recherche (commande *Seek*).
7	Type de support inconnu.
8	Secteur non trouvé.
9	Plus de papier sur l'imprimante.
0AH	Erreur d'écriture.
0BH	Erreur en lecture.
0CH	Autre erreur (*General Failure*).

Contenu	Contenu de la pile utilisateur
IP, CS Drapeaux	Sauvegarde du contexte lors de l'appel à INT 24H.
AX, BX, CX, DX SI, DI, BP, DS, ES	Registres utilisateur sauvegardés par le DOS après appel au DOS par INT 21H.
IP, CS Drapeaux	Sauvegarde du contexte lors de l'appel au DOS par le programme utilisateur.

Si une instruction IRET est exécutée, MSDOS réagit en fonction de la valeur du registre AL : 0 (*Ignore*), 1 (*Retry*), 2 (*Terminate* à l'aide de INT 23H). Le traitement d'erreur doit s'abstenir de faire appel au DOS via INT 21H (les fonctions 1 à 0CH sont cependant utilisables). Si l'on désire effectuer ce traitement d'erreur, le programme doit "vider" la pile en y laissant le contexte permettant d'effectuer une instruction IRET ramenant au programme ayant appelé le DOS. L'état du DOS est dit *unstable*, jusqu'à ce qu'un appel à une fonction de numéro supérieur à 0CH soit effectué.

VI.2. Les entrées/sorties de caractères

INT 21H fonction 1	1.0 Lecture clavier avec écho (→ 3FH)
	AH 1
	AL code caractère
	S'il s'agit de <CTRL C>, INT 23H est exécuté

INT 21H fonction 2	1.0 Affichage caractère (→ 40H)
	AH 2 DL code caractère
	S'il s'agit de <CTRL C> INT 23H est exécuté.

INT 21H fonction 3	1.0 Lecture caractère sur entrée auxiliaire (→ 3FH)
	AH 3
	AL code caractère
	S'il s'agit de <CTRL C> INT 23H est exécuté.

INT 21H fonction 4	1.0 Envoi caractère sur sortie auxiliaire (→ 40H)
	AH 4 DL code caractère
	S'il s'agit de <CTRL C> INT 23H est exécuté.

INT 21H fonction 5	1.0 Envoi caractère sur imprimante (→ 40H)
	AH 5 DL code caractère
	Si <CTRL C> sur la console, INT 23H est exécuté.

INT 21H fonction 6	1.0 Entrée/sortie directe sur console
	AH 6 DL – si FFH positionne le flag Z – si ≠ FFH le caractère dans DL est affiché
	ZF si DL valait FFH, ZF=0 signifie qu'un caractère a été tapé et son code est fourni par AL. Si ZF=1, aucun caractère n'a été tapé. AL code du caractère
	Si <CTRL C> sur la console, INT 23H est exécuté.

INT 21H fonction 7	1.0 Entrée directe sur console
	AH 7
	AL code du caractère
	Attente de caractère, sans d'écho ni test de <CTRL C>.

INT 21H fonction 8	1.0 Lecture clavier
	AH 8
	AL code du caractère
	Attente de caractère avec test de <CTRL C> et pas d'écho.

INT 21H fonction 9	1.0 Affichage chaîne de caractères (→ 40H)
	AH 9 DS:DX adresse de la chaîne
	La chaîne doit se terminer avec un caractère "$".

INT 21H fonction 0AH	1.0 (→ 3FH)
	AH 0AH DS:DX adresse du tampon
	Le <CTRL C> est traité et le tampon contient : – octet 1 : nombre max. de caractères dans le tampon (CR inclus), – octet 2 : nombre de caractères tapés (sans le CR), – puis les caractères.

INT 21H fonction 0BH	1.0 Test de l'état du clavier
	AH 0BH
	AL FF s'il y a des caractères dans le *buffer* 0 si le *buffer* est vide
	<CTRL C> est traité.

INT 21H fonction 0CH	1.0 Vide le *buffer* clavier et lecture clavier
	AH 0CH AL fonction de lecture MSDOS 1, 6, 7, 8, A si AL prend une autre valeur le buffer est simplement vidé.
	AL si 0, la valeur en entrée était ≠ 1, 6, 7, 8, A

VI.3. Les accès disque

INT 21H fonction 0DH	1.0 Mise à jour disques
	AH 0DH
	Cette fonction est utilisée pour s'assurer que les tampons disque et le contenu des disques sont bien identiques. Tous les tampons modifiés sont reécrits, mais les répertoires ne sont pas mis à jour (il faut fermer les fichiers pour que cela soit fait). Les programmes traitant <CTRL C> doivent utiliser cette fonction.

INT 21H fonction 0EH	1.0 Sélection disque
	AH 0EH DL numéro de disque (0 pour A, 1 pour B, ...)
	AL nombre de lecteurs logiques

INT 21H fonction 19H	1.0 Trouver le disque courant
	AH 19H
	AL numéro du disque courant (0 pour A, 1 pour B, ...)
	Au lancement d'un programme, l'adresse de DTA est fixée à l'adresse 80H du PSP.

INT 21H fonction 1AH	1.0 Définir l'adresse de transfert disque
	AH 1AH DS:DX adresse de transfert
	Les transferts ne peuvent s'étendre d'un segment à un autre.

INT 21H fonction 1BH	2.0 Obtention informations disque courant (→ 36H)
	AH 1BH
	AL nombre de secteurs par granule DS:BX adresse du descripteur de support (F8H, ...) DX nombre de granules
	Les transferts ne peuvent s'étendre d'un segment à un autre. Dans la V.1, DS:BX donnait l'adresse de la copie de la FAT en mémoire.

INT 21H fonction 1CH	2.0 Obtention informations disque (→ 36H)
	AH 1CH DL unité de disque (0 pour le lecteur courant, 1 pour A, ...)
	AL nombre de secteurs par granule DS:BX adresse du descripteur de support DX nombre de granules
	Les transferts ne peuvent s'étendre d'un segment à un autre.

INT 21H fonction 1FH	5.0 Lecture DBP du disque par défaut
	AH 1FH
	AL 0FFH lecteur invalide ou erreur disque AL 0 DS:BX = adresse du tampon d'information disque

INT 21H fonction 2FH	2.0 Obtention de l'adresse de transfert disque
	AH 2FH
	ES:BX adresse de transfert disque

INT 21H fonction 32H	5.0 Lire adresse du tampon des paramètres disque
	AH 32H DL numéro de disque (0=défaut, 1=A, ...)
	DS:BX adresse du bloc des paramètres disque

INT 21H fonction 36H	2.0 Obtention espace disque disponible
	AH 36H DL numéro de lecteur (0 pour le lecteur courant, 1 pour A, ...)
	BX nombre de granules libres DX nombre de granules du support désigné CX nombre d'octets par secteur AX nombre de secteurs par granule ou 0FFFFH si le numéro de lecteur est invalide

Exemple d'utilisation des informations sur l'occupation du disque : on exécute les fonctions 1CH et 36H sur un disque dur dont on examine aussi le secteur de *boot*. On obtient les informations suivantes :

– fonction 1CH :

```
al = 4 (4 secteurs par granule)
DS:BX = 135:598 (pointeur vers l'information F8)
DX = 281D (10269) granules
```

– fonction 36H :

```
ax = 4 (4 secteurs par granule)
BX = 16F ((367) granules libres
CX = 200 octets par secteur
DX = 281D (10269) granules
```

– examen du secteur de boot :

```
EB 3C 90 4D 53 44 4F 53-35 2E 30 00 02 04 01 00
02 00 02 E7 A0 F8 29 00-11 00 18 00 11 00 00 00
...
```

Le secteur de boot indique qu'il y a A0E7 (41191_{10}) secteurs au total. Ce nombre de secteurs inclue le secteur de *boot*, les FATs' et le répertoire racine. Le nombre de secteurs utiles est de :

A0E7-1-(2*29)-20=A074 (41076_{10})

soit en nombre de 281D=10269_{10} granules de 4 secteurs.

VI.4. L'accès aux fichiers

INT 21H fonction 0FH	1.0 Ouverture fichier (→ 3DH)
	AH 0FH DS:DX adresse d'un FCB non ouvert
	AL 0 si fichier trouvé FFH si fichier non trouvé
	– Si le numéro de lecteur est 0 cela indique le disque par défaut. – Le champ *bloc courant* est mis à 0. – La taille de l'enregistrement est mise à 128 par défaut. – La taille du fichier, les date et heure de dernière modification sont mises à jour dans le FCB.

INT 21H fonction 10H	1.0 Fermeture fichier (→ 3EH)
	AH 10H DS:DX adresse d'un FCB ouvert
	AL 0 si fichier trouvé ou FFH si fichier non trouvé

INT 21H fonction 13H	1.0 Effacement de fichier (→ 41H)

	AH 13H DS:DX adresse d'un FCB non ouvert
	AL 0 si fichier trouvé ou FFH si fichier non trouvé

INT 21H fonction 14H	1.0 Lecture séquentielle (→ 3FH)
	AH 14H DS:DX adresse d'un FCB ouvert
	AL 0 lecture effectuée avec succès 1 fin de fichier atteinte 2 zone DTA trop petite 3 fin de fichier atteinte avec un enregistrement incomplet
	L'enregistrement défini par "numéro de bloc courant" et "numéro d'enregistrement courant" est chargé dans la zone de transfert DTA.

INT 21H fonction 15H	1.0 Ecriture séquentielle (→ 40H)
	AH 15H DS:DX adresse d'un FCB ouvert
	AL 0 écriture effectuée avec succès 1 disque plein 2 zone DTA trop petite
	L'enregistrement "numéro de bloc courant"/"numéro d"enregistrement courant" est écrit à partir du contenu de la zone de transfert DTA.

INT 21H fonction 16H	1.0 Création fichier (→ 3CH)
	AH 16H DS:DX adresse d'un FCB non ouvert
	AL 0 le répertoire est vide FFH pas de répertoire vide disponible

INT 21H fonction 17H	1.0 Renommer un fichier (→ 56H)
	AH 17H DS:DX adresse du FCB modifié
	AL 0 fichier trouvé FFH fichier non trouvé ou existe déjà

INT 21H fonction 21H	1.0 Lecture en accès direct (→ 3FH)
	AH 21H DS:DX adresse d'un FCB ouvert

	AL 0 lecture effectuée avec succès 1 fin de fichier atteinte 2 zone DTA trop petite 3 fin de fichier atteinte avec un enregistrement incomplet
	Le numéro de bloc courant et le numéro d'enregistrement courant sont mis à jour en accord avec le champ *numéro d'enregistrement relatif.*

INT 21H fonction 22H	1.0 Ecriture en accès direct (→ 40H)
	AH 22H DS:DX adresse d'un FCB ouvert
	AL 0 écriture effectuée avec succès 1 disque plein 2 zone DTA trop petite
	Le numéro de bloc courant et le numéro d'enregistrement courant sont mis jour en accord avec le champ *numéro d'enregistrement relatif.*

INT 21H fonction 23H	1.0 Taille de fichier (→ 42H)
	AH 23H DS:DX adresse d'un FCB non ouvert
	AL 0 fichier trouvé FFH fichier non trouvé
	Champ *numéro d'enregistrement relatif* = nombre d'enregistrements du fichier.

INT 21H fonction 24H	1.0 Fixe le numéro d'enregistrement relatif (→ 42H)
	AH 24H DS:DX adresse d'un FCB non ouvert
	Le champ *numéro d'enregistrement relatif* est calculé à partir du numéro de bloc courant et d'enregistrement courant.

INT 21H fonction 27H	1.0 Lecture de blocs en accès direct (→ 3FH et 42H)
	AH 27H DS:DX adresse d'un FCB ouvert CX nombre de blocs à lire
	AL 0 lecture effectuée avec succès 1 fin de fichier atteinte 2 fin de segment atteinte 3 fin de fichier avec enregistrement partiel CX nombre de blocs effectivement lus
	Si AL = 3 l'enregistrement lu est complété avec des caractères NUL.

INT 21H fonction 28H	1.0 Ecriture de blocs en accès direct (→ 3FH et 42H)
	AH 28H DS:DX adresse d'un FCB ouvert CX nombre de blocs à lire
	AL 0 écriture effectuée avec succès 1 fin de fichier atteinte 2 fin de segment atteinte CX nombre de blocs effectivement écrits

INT 21H fonction 29H	1.0 Analyse nom de fichier
	AH 29H AL contrôle d'analyse *bit* 0 à 0 analyse arrêtée dès qu'on trouve un séparateur 1 les séparateurs sont ignorés *bit* 1 à 0 : si pas de lecteur, N° de lecteur mis à 0 dans le FCB 1 : si pas de lecteur, numéro inchangé dans le FCB *bit* 2 à 0 : si pas de nom de fichier, le nom de fichier n'est pas changé dans le FCB 1 : si pas de nom de fichier, on met 8 "espaces" dans le nom du fichier dans le FCB *bit* 3 à 0 : si pas d'extension, l'extension inchangée dans FCB 1 : si pas d'extension, on met 3 "espaces" à la place DS:SI adresse de la chaîne à analyser ES:DI adresse FCB non ouvert
	AL 0 pas de caractères de remplacement 1 existence de caractères de remplacement FFH identificateur non valide de disque DS:SI adresse du premier octet suivant la chaîne à analyser ES:DI adresse FCB non ouvert
	La chaîne est analysée pour trouver un nom de fichier. Un FCB non ouvert est créé si l'on en trouve un.

VI.5. Les fonctions de l'horloge/calendrier

INT 21H fonction 2AH	1 Obtention date
	AH 2AH
	CX année (1980-2099) DH mois (1-12) DL jour (1-31) AL jour de la semaine (0 pour dimanche, ..., 6 pour samedi)

INT 21H fonction 2BH	1 Définition date
	AH 2BH CX année (1980-2099) DH mois (1-12) DL jour (1-31)
	AL 0 pour date valide 0FFH pour date invalide

INT 21H fonction 2CH	1 Obtention heure
	AH 2CH
	CH heure (0-23) CL minutes (0-59) DH secondes (0-59) DL centièmes de seconde (0-99)

INT 21H fonction 2DH	1 Définition heure
	AH 2DH CH heure (0-23) CL minutes (0-59) DH secondes (0-59) DL centièmes de seconde (0-99)
	AL 0 pour heure valide 0FFH pour heure invalide

VI.6. Accès aux indicateurs du DOS

Un certain nombre de fonctions d'accès aux variables internes du DOS ne sont pas documentées. La seule garantie que l'on a repose sur le nombre et l'importance des éditeurs de logiciels qui ont pu les utiliser. Nous avons déjà cité le cas de la gestion de l'espace mémoire à l'aide d'une liste chaînée.

INT 21H fonction 2EH	1.0 Définition du drapeau de vérification
	AH 2EH AL 0 pas de vérification 1 vérification

INT 21H fonction 3300H	2.0 Lecture du drapeau de contrôle de <CTRL C>
	AH 3300H
	DL état du test : 0 si OFF, 1 si ON
	Normalement le test de <CTRL C> n'est effectué que sur les fonctions 1 à 0CH. La fonction 33H permet d'étendre ce test aux autres fonctions.

INT 21H fonction 3301H	2.0 Initialisation du drapeau de vérification
	AX 3301H DL 0 test inhibé 1 test validé
	Normalement le test de <CTRL C> n'est effectué que sur les fonctions 1 à 0CH. La fonction 33H permet d'étendre ce test aux autres fonctions.

INT 21H fonction 3305H	2.0 Numéro du disque de démarrage
	AX 3305H
	DL numéro de disque de lancement (1=A, 2=B, ...)

INT 21H fonction 3306H	5.0 Lire version DOS
	AX 3306H
	BL Code majeur BH Code mineur DL Numéro de version de révision (3 bits de poids faible) DH Drapeaux (8 pour MSDOS en ROM, 10H pour MSDOS en HMA)

INT 21H fonction 3701H	
	AX 3701H
	Set Switch-AR

INT 21H fonction 52H	Lire adresse de la table des adresses des listes chaînées
	AH 52H
	ES:BX adresse table des adresses des listes chaînées (MCB, DD, ...)
	Les fonction SUBST et JOIN, notamment, utilisent cette fonction. Les pointeurs permettent d'accéder aux tables : – *Memory Control Blocks*, – Chaîne des *Device Drivers*, – Structure de répertoire courant (*Current Directory Structure*), – *System File Table*, ...

INT 21H fonction 53H	
	AH 53H

	Translate BIOS Parameters Block

INT 21H fonction 54H	2 Donne l'état du drapeau de vérification
	AH 54H
	AL drapeau de vérification

INT 21H fonction 55H	
	AH 55H
	Create New PSP

INT 21H fonction 5D00H	
	AX 5D00H
	Server Function Call

INT 21H fonction 5D06H	
	AX 5D06H
	Get DOS Swappable Disk Area

INT 21H fonction 5D0AH	
	AX 5D0AH
	Set Extended-Error Information

VI.7. Le contrôle de fin d'exécution de programme

INT 21H fonction 0	1.0 Fin d'exécution (→ 4CH)
	AH 0 CS segment du PSP courant
	Cet appel est équivalent à INT 20H. Tous les fichiers dont la taille a changé doivent être fermés avant d'appeler cette fonction.

INT 21H fonction 31H	2.0 Conserve le processus courant en mémoire

AH	31H
AL	code de sortie récupérable par le processus père
DX	taille mémoire à réserver en nombre de paragraphes

INT 21H fonction 34H

2.0	Lire adresse du drapeau InDOS
AH	34H
DS:BX	adresse d'un "sémaphore" (octet)
Il s'agit plus d'un compteur que d'un sémaphore. Lorsque ce drapeau est différent de 0, MSDOS est en train d'exécuter une fonction.	

INT 21H fonction 4B00H

2.0	Charge et exécute un programme (*)
AX	4B00H
DS:DX	adresse "chemin d'accès/nom de programme" (terminée par 0)
ES:BX	adresse d'un bloc de paramètres
AL	0 chargement et exécution de programme
CY=1	AX = 1 fonction incorrecte AX = 10 environnement incorrect AX = 11 format incorrect AX = 2 fichier non trouvé AX = 8 taille mémoire insuffisante
Les *handles* des fichiers ouverts sont accessibles dans le processus fils (à moins de le demander explicitement à l'ouverture du fichier-fonction 3DH). Le processus père peut garder tout contrôle sur ceux-ci.	

En version 2.X, le système ne sauvegarde que les registres CS et IP. Les fichiers désignés sont de type .EXE ou .COM. Pour un fichier de type .BAT ou pour exécuter des commandes internes, il faut appeler COMMAND.COM par la chaîne de commande (terminée par un <retour chariot>) :

```
'COMMAND.COM /c <nom_de_fichier.BAT>',13 ou
'COMMAND.COM /c <commande_interne>',13
```

INT 21H fonction 4B01H

2.0	Charge un programme en mémoire
AX	4B01H
Le programme chargé n'est pas lancé.	

INT 21H fonction 4B03H

2.0	Charge *Overlay*
AX	4B03H
DS:DX	adresse nom de programme (terminé par NUL)
ES:BX	adresse d'une structure de chargement d'*overlay*
CY=1	AX conteint un code d'erreur (1 à 5, 8 ou A)

Le programme chargé n'est pas lancé. La structure d'*overlay* contient : – mot : segment de l'*overlay* – mot : facteur de relocation

INT 21H fonction 4B05H

5.0	Prépare un programme pour exécution
AX	4B05H
DS:DX	adresse d'une structure d'exécution
Cette fonction prépare en particulier le numéro de version pour SETVER. La structure d'exécution contient : – mot réservé – mot pour drapeaux de type – adresse du nom du programme (terminé par NUL) – segment du PSP du nouveau programme – CS:IP du nouveau programme – double mot : taille du programme, PSP compris Cette fonction désactive la ligne A20. Si ce programme, qui est utilisé lorsqu'on déroute la fonction 'charge et exécute un programme', utilise la HMA, il doit revalider la ligne A20.	

INT 21H fonction 4CH

2	Fin d'exécution de processus (*)
AH	4CH
AL	code de retour exploitable par le processus père (fonction 04DH)

VI.8. L'accès aux vecteurs d'interruption

INT 21H fonction 25H

1.0	Définition adresse de traitement d'interruption
AH	25H
AL	numéro d'interruption
DS:DX	adresse du programme de traitement

INT 21H fonction 35H

2.0	Récupération d'adresse de traitement d'interruption
AH	35H
AL	numéro d'interruption
ES:BX	adresse du programme de traitement de l'interruption

VI.9. Fonctions d'accès aux répertoires

INT 21H fonction 39H

2.0	Création sous-répertoire (*)

AH	39H
DS:DX	adresse chaîne de caractères (terminée par NUL) du chemin d'accès
CY=1	AX=3 chemin non valide AX=5 le répertoire ne peut être créé

INT 21H fonction 3AH	2.0	Effacement de répertoire (*)
	AH	3AH
	DS:DX	adresse du chemin d'accès (terminé par NUL)
	CY=1	AX=3 chemin non valide AX=5 le répertoire ne peut être créé AX=16 répertoire courant

INT 21H fonction 3BH	2.0	Changement de répertoire (*)
	AH	3BH
	DS:DX	adresse chaîne de caractères (terminée par NUL) du chemin d'accès
	CY=1	AX=3 chemin non valide

INT 21H fonction 47H	2.0	Fournit le répertoire courant
	AH	47H
	DS:SI	adresse d'une zone mémoire de 64 octets
	DL	numéro de lecteur (1 pour A, 2 pour B, ...)
	CY=1	AX = 15 numéro de lecteur non correct

VI.10. Accès aux fichiers par numéros logiques

La méthode d'accès aux fichiers utilisant des numéros logiques devra être préférée à celle utilisant les blocs de contrôle de fichiers (*FCB*).

INT 21H fonction 3CH	2.0	Création de fichier (*)
	AH	3CH
	DS:DX	adresse de la chaîne (terminée par NUL) du chemin d'accès
	CX	bit 0 à 1 : lecture seulement bit 1 à 1 : fichier caché bit 2 à 1 : fichier système
	CY=1	AX=3 chemin non valide AX=4 trop de fichiers ouverts AX=5 le fichier ne peut être créé
	CY=0	AX=numéro logique (*handle*)
	Si le fichier existait déjà, il est écrasé, sinon il est créé.	

INT 21H fonction 3DH	2.0	Ouverture de fichier (*handle*) (*)
	AH	3DH
	AL	*bits* 0-2 autorisation en lecture/écriture : – 000 *read Only* – 001 *write Only* – 010 *read/Write* *bits* 3 à 0 *bits* 4-6 : – 000 seul le prog. actuel peut accéder au fichier en mode FCB – 001 seul le programme actuel peut accéder au fichier – 010 un autre prog. peut accéder au fichier en lecture seulement – 011 un autre prog. peut accéder au fichier en écriture seulement – 100 un autre prog. peut accéder au fichier bit 7 0 le prog. fils peut accéder au fichier 1 seul le programme actuel peut accéder au fichier
	DS:DX	adresse du nom de fichier (terminé par un octet à 0)
	CY=1	AX=1 pas de logiciel de partage de fichier AX=2 fichier non trouvé AX=3 chemin non trouvé ou fichier n'existe pas AX=4 trop de fichiers ouverts AX=5 accès refusé (violation de type ou de protection) AX=12 accès non valide
	CY=0	AX=numéro logique (*handle*)

INT 21H fonction 3EH	2.0	Fermeture de fichier (*handle*)
	AH	3EH
	BX	numéro logique (*handle*) de fichier
	CY=1	AX=6 numéro non valide

INT 21H fonction 3FH	2.0	Lecture généralisée sur fichier (*)
	AH	3FH
	DS:DX	adresse d'un tampon
	CX	nombre d'octets à lire
	BX	numéro logique du fichier ou périphérique (*handle*)
	CY=1	AX = 5 accès refusé AX = 6 numéro logique invalide
	CY=0	AX = nombre d'octets lus

INT 21H fonction 40H	2.0	Ecriture généralisée sur fichier (*)

AH	40H
DS:DX	adresse d'un tampon
CX	nombre d'octets à écrire
BX	numéro logique du fichier ou périphérique (*handle*)
CY=1	AX = 5 accès refusé
	AX = 6 numéro logique invalide
CY=0	AX = nombre d'octets écrits

INT 21H fonction 41H

2.0	Effacement fichier (*)
AH	41H
DS:DX	adresse de la chaîne "nom de fichier" (le caractère de fin a pour code 0)
CY=1	AX = 5 accès refusé
	AX = 2 fichier non trouvé

INT 21H fonction 42H

2.0	Modification de pointeur dans un fichier (*)
AH	42H
CX-DX	déplacement en octets (sur 32 bits)
AL	0 déplacement à partir du début du fichier
	1 déplacement à partir de la position courante
	2 déplacement à partir de la fin du fichier
BX	numéro logique (*handle*)
CY=1	AX = 6 numéro logique invalide
	AX = 1 fonction (AL) non valide
CY=0	DX-AX nouveau pointeur
Lorsque AL est égal à 0, on a un mode d'accès direct. Le numéro du premier octet du fichier est 0.	

INT 21H fonction 45H

2.0	Duplication de pointeur sur fichier (*)
AH	45H
BX	numéro logique (*handle*)
CY=1	AX = 6 numéro logique non correct
	AX = 4 trop de fichiers ouverts
CY=0	AX = nouveau numéro logique

INT 21H fonction 46H

2.0	Duplication forcée de pointeur sur fichier (*)
AH	46H
BX	numéro logique (*handle*)
CX	nouveau numéro logique
CY=1	AX = 6 numéro logique non correct
	AX = 4 trop de fichiers ouverts

INT 21H fonction 56H

2.0	Renomme dans un autre répertoire
AH	56H
DS:DX	adresse d'une chaîne "chemin d'accès" valide
ES:DI	adresse d'une chaîne "nouveau chemin d'accès"
CY=1	AX = 2 fichier non trouvé
	AX = 17 support différent
	AX = 5 accès refusé
Le support doit être le même	

INT 21H fonction 5AH

3.0	Création de fichier temporaire
AH	5AH
CX	attribut fichier temporaire
DS:DX	adresse répertoire
CY=0	AX=*handle*
DS:DX	adresse nom de fichier complet
CY=1	AX=code erreur (3 = chemin non trouvé, 5 = accès refusé)

INT 21H fonction 67H

3.3	Augmente le nombre de *Handles*
AH	67H
Increase File Handle (de 20 à 65535).	

INT 21H fonction 68H

3.3	Sauve les *Buffers* sur disque
AH	68H
BX	*Handle*
CY=1	AX = code erreur
Force l'écriture des tampons dans les fichiers correspondants.	

INT 21H fonction 6CH

4.0	Création, ouverture et mise à jour
AX	6C00H
BX	mode d'accès
bits 0-2	000: fichier *Read-Only*
	001: fichier *Write-Only*
	010: fichier accessible en lecture/écriture
bits 3	0
bits 4-6	000: accès réservé au process courant
	001: accès réservé au process courant
	010: accès aurorisé en lecture aux autres programmes
	011: accès aurorisé en écriture aux autres programmes
	100: accès aurorisé en lecture/écriture aux autres programmes

	bits 7 0: les programmes enfants peuvent accéder au fichier *bits* 8-12 0 *bits* 13 0: si erreur appeler INT 24H 1: si erreur sortir avec CY et AX *bits* 14 1: mise à jour répertoire à chaque écriture *bits* 15 0 CX attribut (*bits* 0:*Read-only*, 1:caché, 2:fichier système, 5:archive) DX mode opératoire du DOS (DL=xxxxyyyy) yyyy=0000 sortir avec code erreur si fichier n'existe pas encore yyyy=0001 créer le fichier xxxx=0000 sortir avec code erreur si fichier existe déjà xxxx=0001 ouvrir le fichier existant DS:SI adresse nom du fichier (terminé par 0)
	CY=1 AX = code erreur (1, 2, 3, 4, 5 ou 12) CY=0 AX = *handle* et CX état
	Extended Create/Open file

VI.11. L'accès aux informations sur les fichiers

INT 21H fonction 4300H	2.0 Lecture attributs fichier (*)
	AX 4300H DS:DX adresse chaîne "nom de fichier" (terminée par un octet à 0) CX attributs du fichier (si AL = 1)
	CY=1 AX = 3 chemin d'accès incorrect AX = 1 fonction (AL) invalide AX = 2 fichier non trouvé AX = 5 accès refusé CY=0 CX = attributs si fonction (AL) = 0

INT 21H fonction 4301H	2.0 Modification attributs fichier (*)
	AX 4301H DS:DX adresse chaîne "nom de fichier" (terminée par un octet à 0) CX attributs du fichier (si AL = 1)
	CY=1 AX = 3 chemin d'accès incorrect AX = 1 fonction (AL) invalide AX = 2 fichier non trouvé AX = 5 accès refusé

INT 21H fonction 5700H	2.0 Lire date/heure fichier
	AX 5700H BX numéro logique

	CY=1 AX = 1 fonction incorrecte AX = 6 numéro logique incorrect CY=0 CX = heure (sous-fonction 0) DX = date (sous-fonction 0)

INT 21H fonction 5701H	2.0 Ecrire date/heure fichier
	AX 5701H BX=numéro logique, CX=heure, DX=date
	CY=1 AX = 1 fonction incorrecte AX = 6 numéro logique incorrect

VI.12. L'accès aux *Device Drivers*

INT 21H fonction 44H	2.0 Contrôle sur entrées/sorties généralisées (*)
	AH 44H AL=fonction – 0 lecture d'informations sur unité d'entrée/sortie – 1 mise à jour informations sur unité d'entrée/sortie – 2 lecture de CX caractères sur unité d'entrée/sortie – 3 écriture de CX caractères sur unité d'entrée/sortie – 4 lecture de CX caractères sur unité de disque (unité BL) – 5 écriture de CX caractères sur unité de disque (unité BL) – 6 lecture d'état en lecture – 7 lecture d'état en écriture BX numéro logique (*handle*) BL numéro de lecteur (si AL=4,5) – 0 pour lecteur courant, – 1 pour A, 2 pour B, ... DS:DX adresse des données ou d'un tampon (fonctions 2, 3, 4, 5) DX informations sur unité d'entrée/sortie (fonction 1) CX nombre d'octets à lire ou écrire
	CY=1 AX = 1 fonction (AL) invalide AX = 5 accès refusé AX = 13 données incorrectes AX = 6 numéro logique incorrect CY=0 AX = nombre d'octets transmis (fonctions 2, 3, 4, 5) AL = 0 (fonctions 6 et 7) si unité ou fichier prêt AL = 0FFH (6 et 7) si unité ou fichier non prêt DX informations sur unité d'entrée/sortie (fonction 0)

INT 21H fonction 44H	3.0 Contrôle sur entrées/sorties généralisées (*)

AH	44H
AL	– 8 test d'amovabilité support – 0BH fixer le nombre de tentatives d'accès réseau
BX	numéro logique (*handle*) (si AL=0AH) nombre de répétitions (si AL=0BH)
BL	numéro de lecteur (si AL=8) – 0 pour lecter courant, – 1 pour A, 2 pour B, ...
CX	délai entre deux tentatives d'accès (si AL=0BH)
CY=1	AX = 1 fonction (AL) invalide AX = 15 périphérique inconnu AX = 6 numéro logique incorrect
CY=0	AX = 0 support non amovible (fonctions 8) AX = 1 support amovible (fonctions 8) – *bit* 12 = 0 périphérique local – *bit* 12 = 0 périphérique distant

INT 21H fonction 44H

3.1	Contrôle sur entrées/sorties généralisées (*)
AH	44H
AL	code fonction – 9 test de périphérique distant – 0AH test *handle* distant
BX	numéro logique (*handle*) (si AL=0AH)
BL	numéro de lecteur (si AL=9), 0lecteur courant, 1=A, 2=B, ...
CY=1	AX = 1 fonction (AL) invalide AX = 15 périphérique inconnu AX = 6 numéro logique incorrect
CY=0	DX = attribut du périphérique (fonction 9) – *bit* 12 = 0 périphérique local – *bit* 12 = 0 périphérique distant DX = code IOCTL (fonction 0AH) – *bit* 15 = 0 périphérique local – *bit* 15 = 0 périphérique distant

Constitution de DX pour les sous-fonctions 0 et 1 :

bit	
0	1 si l'unité est la console (en entrée)
1	1 si l'unité est la console (en sortie)
2	1 si l'unité est NUL
3	1 si l'unité est l'unité d'horloge
4	1 si l'unité est une unité spéciale
5	1 si l'unité est une unité en mode "caractères" 0 si l'unité est en mode "blocs"

6	0 si l'unité possède une signalisation de fin de fichier
7	1 si l'unité est une unité d'entrée/sortie (bits 0 à 6 utilisés) 0 si l'unité est un fichier disque (les bits 8 à 15 sont à 0)
8 à 13	réservés
14	0 l'unité ne traite pas les chaînes de contrôle lors du traitement des sous-fonctions 2 et 3 1 si l'unité peut traîter des chaînes de contrôle (AL=2 ou 3)
15	réservé

INT 21H fonction 440CH

	IOCTL générique (périphériques caractères)
CL	45H (V3.3) initialise compteur d'itérations 4AH (V3.3) sélectionne code de page 4CH (V3.3) début préparation de code de page 4DH (V3.3) fin préparation de code de page 5FH (V4.0) initialise mode d'affichage 65H (V3.3) lit compteur d'itérations 6AH (V3.3) lit code de page sélectionné 6BH (V3.3) demande liste des codes de page 7FH (V4.0) lit le mode d'affichage

INT 21H fonction 440DH

	IOCTL générique (périphériques blocs)
CL	40H (V3.2) initialise paramètres de périphériques 41H (V3.2) écrit piste sur lecteur logique 42H (V3.2) formatte piste sur lecteur logique 46H (V4.0) initialise identificateur de support 60H (V3.2) lit paramètres de périphériques 61H (V3.2) lit piste sur disque logique 62H (V3.2) vérifie piste sur disque logique 66H (V4.0) lit identificateur de support 68H (V5.0) lit le type de support

INT 21H fonction 440EH

3.2	Renvoie correspondance des lecteurs logiques
BL	numéro de lecteur (0=lecteur par défaut, 1=A, ...)
CY=0	AL=numéro du lecteur actif du lecteur physique correspondant
CY=1	numéro d'erreur dans AX : 0001 numéro de fonction erroné 0005 accès refusé 000F numéro de lecteur erroné

INT 21H fonction 440FH	3.2 Initialise correspondance des lecteurs logiques
	BL numéro de lecteur (0=lecteur par défaut, 1=A, ...)
	CY=0 AL=numéro du lecteur actif du lecteur physique correspondant CY=1 numéro d'erreur dans AX : 0001 numéro de fonction erroné 0005 accès refusé 000F numéro de lecteur erroné

INT 21H fonction 4410H	5.0 Vérification fonction *Handle* IOCTL
	BX *handle de périphérique* CH catégorie de périphérique 01 ligne série 03 écran 05 sortie parallèle CL fonction à vérifier (45H ou 65H)
	CY=1 AX=0001 numéro de fonction erroné AX=0005 accès refusé
	Cette fonction permet de savoir si une fonction IOCTL est bien mise en œuvre par le *driver* de périphérique indiqué par *handle*.

INT 21H fonction 4411H	5.0 Vérification fonction périphérique IOCTL
	BL numéro de lecteur (0=lecteur par défaut, 1=A, ...) CH 8 (catégorie) CL à vérifier : (40H,41H,42H,46H,60H,61H,62H,66H,68H)
	CY=1 AX = 1 numéro de fonction erroné AX = 5 accès refusé AX = F numéro de lecteur erroné
	Cette fonction permet de savoir si une fonction IOCTL est bien mise en œuvre pour le lecteur indiqué.

VI.13. Les fonctions de gestion de la mémoire

INT 21H fonction 48H	2 Allocation de mémoire
	AH 48H BX taille mémoire requise (en paragraphes)
	CY=1 AX = 8 pas assez de mémoire AX = 7 violation d'espace mémoire BX = taille mémoire que l'on peut allouer CY=0 AX:0 adresse de la mémoire allouée

INT 21H fonction 49H	2 Libération d'espace mémoire
	AH 49H BX nombre de paragraphes à libérer ES segment de la zone à libérer
	CY=1 AX = 9 le segment donné dans ES n'est pas correct AX = 7 violation d'espace mémoire
	Libération de ce qui a été alloué par la fonction 48H.

INT 21H fonction 4AH	2 Modification d'allocation de mémoire
	AH 4AH ES segment de la zone BX taille mémoire requise (en paragraphes)
	CY=1 AX = 9 le segment donné dans ES n'est pas correct AX = 7 violation d'espace mémoire AX = 8 pas assez de mémoire BX = taille mémoire qui peut être allouée

INT 21H fonction 5800H	3.0 Lire principe de répartition mémoire
	AX 5800H
	CY=1 AX=1 code fonction inconnu CY=0 AX=0 recherche en partant des adresses basses AX=1 recherche selon meilleure méthode AX=2 recherche en partant des adresses hautes

INT 21H fonction 5801H	3.0 Fixer principe de répartition mémoire
	AX 5801H BX principe : – 0 en partant des adresses basses – 1 bloc de taille la plus proche – 2 en partant des adresses hautes – 80H premier bloc en mémoire haute (sinon la recherche s'effectue en mémoire normale) – 81H bloc de taille la plus proche en mémoire haute ... – 82H dernier bloc en mémoire haute ... – 40H premier bloc en mémoire haute – 41H bloc de taille la plus proche en mémoire haute – 42H dernier bloc en mémoire haute

INT 21H fonction 5802H	5.0 Lire lien vers mémoire haute

	AX 5802H
	AL 1 si lien, 0 sinon
	Permet de savoir si un prog. peut allouer des blocs en mémoire haute.

INT 21H fonction 5803H	5.0 Initialise lien vers mémoire haute
	AX 5803H BX 1 si lien, 0 sinon
	CY=1 AX=1 fonction non valide (si DOS=UMB n'a pas été précisée) AX = 7 zone mémoire corrompue

VI.14. Fonctions diverses

INT 21H fonction 30H	2.0 Version DOS
	AH 30H
	AL numéro principal de version AH numéro secondaire de version

INT 21H fonction 38H	2.0 Lire informations liées au pays
	AH 38H DS:DX adresse d'un tampon de 32 octets au moins AL 0 lecture paramètres pour pays courant, ≠ 0 code pays
	si DX=0FFFFH change le code courant par le code donné dans AL

INT 21H fonction 38H	3 Déterminer le pays
	AH 38H DS:DX adresse d'un tampon de 32 octets au moins (si DX=0FFFFH la fonction change le code courant par lecode donné dans AL) AL 1-254 numéro de pays 255 : alors BX doit contenir le code pays
	CY=1 code pays incorrect

Zone de données d'informations sur le pays (les chaînes de caractères (CC) sont toutes terminées par le caractère NUL) :

Déplacement	
0-1	format pour la date et l'heure
2-6	CC symbole monétaire
7-8	CC séparateur pour les milliers
9-10	CC séparateur décimal
11-12	CC séparateur pour la date

13-14	CC séparateur pour l'heure
15	0 symbole monétaire précède le montant sans espace 1 symbole monétaire suit le montant sans espace 2 symbole monétaire précéde le montant avec un espace 3 symbole monétaire suit le montant avec un espace
16	nombre de décimales dans la représentation monétaire
17	format pour l'heure (0-1 /1-24)
18-21	adresse de traitement pour les caractères de code ≥ 80H (particularités nationales : conversion minuscules --> majuscules)
22-23	CC séparateur de listes de données

Le bloc de paramètres a le format suivant :

octet	Contenu (sous fonction 0)
0-1	segment de l'environnement
2-5	adresse complète de la ligne de commande (80H dans le PSP)
6-9	adresse complète du FCB (5CH dans le PSP)
0AH-0DH	adresse complète du FCB (6CH dans le PSP)
	Lorsque l'adresse de l'environnement est 0, le processus fils hérite de l'environnement du processus père

octet	Contenu (sous fonction 3)
0-1	segment de chargement du fichier
2-3	facteur de réimplantation appliqué à l'"image"

INT 21H fonction 4DH	2.0 Récupération état d'un processus fils (*)
	AH 4DH
	AL code de sortie AH 0 fin sur "abort" 1 fin sur <CTRL C> 2 fin sur erreur matérielle 3 fin mais reste résident

INT 21H fonction 6501H	3.3 Lire informations étendues sur pays
	AX 6501H BX code de page (si 0FFFH pays courant) CX taille du tampon DX code du pays ES:DI adresse d'une structure d'information sur le pays
	CY=1 AX = code d'erreur 1 ou 2

INT 21H fonction 6502H	3.3 Lire table des majuscules

AX	6502H
BX	code de page (si 0FFFH pays courant)
CX=5	taille du lien du tampon
DX	code de pays
ES:DI	adresse du tampon
CY=1	AX = code d'erreur (1 ou 2)

Cette table fait correspondre les codes > 128 à leur équivalents en majuscule.
Le lien de tampon contient : – octet d'identification (=2)
– adresse (4 octets) de la table

INT 21H fonction 6504H

3.3	Lire table des majuscules pour noms de fichiers
AX	6504H
BX	code de page (si 0FFFH pays courant)
CX=5	taille du lien du tampon
DX	code de pays
ES:DI	adresse du tampon
CY=1	AX = code d'erreur (1 ou 2)

INT 21H fonction 6505H

3.3	Lire table des caractères pour noms de fichiers
AX	6505H
BX	code de page (si 0FFFH pays courant)
CX=5	taille du lien du tampon
DX	code de pays
ES:DI	adresse du tampon
CY=1	AX = code d'erreur (1 ou 2)

INT 21H fonction 6506H

3.3	Lire table de correspondance
AX	6506H
BX	code de page (si 0FFFH pays courant)
CX=5	taille du lien du tampon
DX	code de pays
ES:DI	adresse du tampon
CY=1	AX = code d'erreur (1 ou 2)

La table contient 256 éléments indiquant l'ordre alphabétique.

INT 21H fonction 6507H

3.3	Lire table des caractères (sur deux octets)
AX	6507H
BX	code de page (si 0FFFH pays courant)
CX=5	taille du lien du tampon
DX	code de pays
ES:DI	adresse du tampon

CY=1	AX = code d'erreur (1 ou 2)

INT 21H fonction 6520H

3.3	Conversion en majuscules
AX	6520H
DL	code du caractère
DL	caractère converti en utilisant la table des majuscules

INT 21H fonction 6521H

3.3	Conversion de chaîne
AX	6521H
CX	longueur de la chaîne
DS:DX	adresse de la chaîne

INT 21H fonction 6522H

3.3	Conversion de chaîne ASCIIZ
AX	6522H
DS:DX	adresse de la chaîne

INT 21H fonction 6601H

3.3	Lire code de page global
AX	6601H
BX	code de page utilisateur
DX	code de page système
CY=1	AX=1

INT 21H fonction 6602H

3.3	Initialise code de page global
AX	6602H
BX	code de page

VI.15. L'accès aux répertoires

INT 21H fonction 11H

1.0	Recherche d'un fichier dans le répertoire (recherche par FCB, première occurence)
(→ 4EH)	
AH	11H
DS:DX	adresse d'un FCB non ouvert
AL	0 si fichier trouvé FFH si fichier non trouvé

INT 21H fonction 12H

1.0	Recherche de la prochaine occurence dans le répertoire
(→ 4FH)	(recherche par FCB)

AH	12H
DS:DX	adresse d'un FCB non ouvert
AL	0 si fichier trouvé FFH si fichier non trouvé
Cette fonction est appelée après la fonction 11H	

INT 21H fonction 4EH

2	Recherche de fichier d'un attribut donné (première occurence)
AH	4EH
DS:DX	adresse de chaîne "chemin d'accès"
CX	attribut de recherche : – 0 recherche des fichiers "normaux" au sens du DOS – ≠0 recherche de tous les fichiers dont l'attribut est donné.
CY=1	AX = 2 chemin non trouvé AX = 18 fichier non trouvé
Cette fonction accepte un chemin d'accès contenant des *wildcards*. A l'adresse de transfert, on trouve un bloc d'information (tableau ci-après). L'adresse de transfert doit avoir été définie au préalable (fonction 1AH).	

octet	Contenu du bloc d'information
0-20	réservés
21	attribut trouvé
22-23	heure
24-25	date
26-27/28-29	taille (poids faible) / (poids fort)
30-42	nom

INT 21H fonction 4FH

2.0	Suite de la recherche
AH	4FH
CY=1	AX = 18 plus de fichiers
Cette fonction fait suite à la fonction 4EH lorsqu'on a des caractères de remplacement dans la nom du fichier. La DTA ne doit pas être modifiée entre le premier appel à la fonction 4EH et les appels à la fonction 4FH.	

VI.16. L'accès au préfixe de segment programme

INT 21H fonction 26H

1.0	Création de PSP (→ 4B00H)
AH	26H
DS	adresse de segment du nouveau PSP

INT 21H fonction 50H

2.0	Définition d'une adresse de PSP
AH	50H

Set PSP Address (accompagné du changement de la DTA, on peut éventuellement commuter d'une tâche à l'autre).

INT 21H fonction 51H

2.0	Lecture adresse de PSP
AH	51H
Get PSP address (remplace la fonction 62H)	

INT 21H fonction 59H

3.0	Lire code étendu d'erreur
AH	59H
BX	0
AX	erreur
BH	cause erreur
BL	conseil
CH	origine erreur

INT 21H fonction 5D0AH

4.0	Etablit les classes d'erreur
AX	5D0AH
DS:SI	adresse d'une structure d'erreur
La structure d'erreur contient : – 8 mots pour les registres AX,BX,CX,DX,SI,DI,DS,ES, – mot réservé – mot : identificateur utilisateur – mot : identificateur de processus	

VI.17. Complément sur les fonctions de gestion de réseau

NetBios (*NETwork Basic IO System*) constitue une couche logicielle qui vient intercepter les appels standard au DOS. Ce logiciel met en œuvre (à peu près ...) les couches 2 à 5 du modèle ISO (le niveau *liaison* ne respecte pas la normalisation ISO car il n'y a pas de contrôle d'erreur, lequel est laissé à la charge de la couche *transport*).

INT	AH	AL	fonction
21H	3DH		ouverture de fichier avec attributs de partage
	44H	9	IOCTL, unité d'E/S redirigée ?
		0AH	IOCTL, numéro logique local ou distant ?
		0BH	IOCTL, modification du compteur d'accès partagé
	59H		lecture code d'erreur étendu
	5AH		création de fichier temporaire
	5BH		création de nouveau fichier

21H	5CH	0	blocage
		1	déblocage
	5EH	0	lecture nom de machine
		2	définition de chaîne de contrôle imprimante
	5FH	2	lecture adresse de liste d'assignation
		3	redirection unité d'E/S vers le réseau
		4	annulation de redirection
2AH	0		test d'installation NETBIOS
	3		lecture d'état d'unité partagée
	4		exécution NETBIOS
	5		lecture des informations ressources réseau
	6		contrôle d'impression sur le réseau
2FH	87	0	ajoût sur test d'installation
		2	ajoût sur test de version
	88	0	test de commande réseau
		3	lecture d'adresse du serveur de messagerie
		4	écriture d'adresse du serveur de messagerie
		9	test de version de réseau

VI.18. Exemples

Exemple de recherche de fichiers et affectation automatique de nom de fichier à partir d'un numéro : le nom du fichier `Nfich$` possède le chemin d'accès `PathImp$` et a pour extension `EXT`. Il commence par les trois caractères `FIC` et est complété par un numéro de trois chiffres.

```
'* Toutes les variables dont le nom commence par i, j, ..., n
'* sont des entiers
DEFINT I-N
Nerr = 0: Nfich = 0
WHILE Nerr = 0
    Nb$ = LTRIM$(STR$(Nfich))
    WHILE LEN(Nb$) < 3: Nb$ = "0" + Nb$: WEND
    Nfich$ = PathImp$ + "\FIC" + Nb$ + ".EXT"
    CALL FileSearch(Nfich$, Nerr)
    Nfich = Nfich + 1
WEND
```

Le programme de recherche de fichier utilisant la fonction 4EH fait appel au DOS à l'aide de la procédure `interruptX` à laquelle on transmet le numéro d'interruption et deux structures pour les registres. Ces deux structures ont pour type `RegTypeX` :

```
TYPE RegTypeX
     ax    AS INTEGER
     bx    AS INTEGER
     cx    AS INTEGER
     dx    AS INTEGER
     bp    AS INTEGER
```



```
     si    AS INTEGER
     di    AS INTEGER
     flags AS INTEGER
     ds    AS INTEGER
     es    AS INTEGER
END TYPE

'*-----------------------------------*
'* Recherche d'un fichier            *
'*    Nom$  = nom du fichier         *
'*    NoErr = numéro d'erreur si <> 0 *
'*-----------------------------------*
SUB FileSearch (Nom$, NoErr)
DIM iRegIn AS RegTypeX, iRegOut AS RegTypeX

'* Le tableau d'entiers iBidon() sert de DTA
' $DYNAMIC
DIM iBidon(64)
'* Définition de la DTA. On transmet à la fonction 1AH
'* l'adresse du tableau
iRegIn.ax = &H1A00: IntNum = &H21
iRegIn.ds = VARSEG(iBidon(0))
iRegIn.dx = VARPTR(iBidon(0))
CALL interruptX(IntNum, iRegIn, iRegOut)

'* Recherche à l'aide de la fonction 4EH
NoErr = 0: IntNum = &H21: iRegIn.ax = &H4E00: iRegIn.cx = 0
'* on complète le nom du fichier avec un caractère NUL
Name$ = Nom$ + CHR$(0): iRegIn.ds = VARSEG(Name$)
iRegIn.dx = SADD(Name$)
CALL interruptX(IntNum, iRegIn, iRegOut)
'* on récupère le drapeau de retenue CY dans l'entier iCarry
iCarry = iRegOut.flags AND 1
IF iCarry = 1 THEN NoErr = iRegOut.ax

ERASE iBidon
END SUB
```

CHAPITRE VII

VII. La gestion de la souris

Le *driver* de gestion de la souris est fourni, soit sous forme d'un *Device Driver* MOUSE.SYS, soit sous forme d'un utilitaire résident MOUSE.COM.

VII.1. Les masques de définition du curseur graphique

Le curseur associé à la souris peut être soit un curseur de type *texte*, soit un curseur de type *graphique*.

En mode graphique, la forme du curseur est définie par un *masque d'écran* et un *masque de curseur*. Chaque masque est constitué de 16 mots de 16 *bits*. Les masques sont utilisés de la façon suivante :

– un ET logique est effectué entre le masque d'écran et l'état des pixels de l'écran,
– puis un OU EXCLUSIF est effectué entre le résultat de l'opération précédente et le masque de curseur.

En moyenne résolution, à chaque pixel est associé soit 2 *bits*, soit 4 *bits* selon le nombre de couleurs du mode d'affichage choisi.

Il existe deux types de curseurs en mode texte :

– un curseur dit "système", qui est constitué d'un bloc clignotant dont on définit la première ligne et la dernière ligne active (fonction 10),
– un curseur dit "logique", qui affecte la forme et l'attribut du caractère qu'il recouvre en fonction de deux masques de 16 *bits* dits "masque d'écran" et "masque de curseur". Le logiciel effectue d'abord un ET logique entre les 16 *bits* du caractère affiché (code + attribut) et le masque d'écran, puis un OU exclusif entre le résultat de cette opération et le masque de curseur.

Au curseur est associé la notion de point actif, ou point *chaud*, qui définit la position du curseur sur l'écran. Pour un curseur de type "texte", la position est donnée sous forme ligne-colonne de texte. Pour un curseur "graphique", la position est donnée sous forme ligne-colonne "pixel".

VII.2. Les fonctions de gestion de la souris

Les fonctions de gestion de la souris sont appelées à partir de l'interruption INT 33H.

INT 33H/0	Initialisation de l'état de la souris
	AX 0
	AX –1 souris installée, 0 souris non installée BX nombre de boutons de la souris

INT 33H/1	Affichage curseur
	AX 1

INT 33H/2	Effacement curseur
	AX 2

INT 33H/3	Acquisition état et position du curseur
	AX 3
	BX état boutons : *bit* 0 bouton gauche (souris à 2 ou 3 boutons) *bit* 1 bouton droit (souris à 2 ou 3 boutons) *bit* 2 bouton milieu (souris à 3 boutons) CX position horizontale DX position verticale
	Le bit d'état est à 0 pour un bouton relevé, à 1 sinon. En mode texte, la position est donnée en pixels, l'écran étant considéré comme étant un écran graphique avec des caractères 8×8, quelque soit le type d'affichage.

INT 33H/4	Positionnement curseur
	AX 4 CX position horizontale DX position verticale

INT 33H/5	Lecture état d'enfoncement des boutons
	AX 5 BX choix du bouton (voir fonction 3)
	AX état du bouton (voir fonction 3) BX nombre de fois où le bouton a été appuyé depuis le dernier appel. CX position horizontale lors de la dernière pression DX position verticale lors de la dernière pression
	Le compteur est remis à zéro après appel de cette fonction.

INT 33H/6	Lecture état de relèvement des boutons
	AX 6 BX choix du bouton (voir fonction 3)

AX,BX,CX,DX : mêmes défintions que pour la fonction 5 de gestion de la pression sur le bouton

Le compteur est remis à zéro après appel de cette fonction

INT 33H/7

Définition abscisses min. et max. pour le curseur	
AX	7
CX, DX	abscisse minima, maxima

Les valeurs de CX et DX sont permutées si DX<CX

INT 33H/8

Définition ordonnées min. et max. pour le curseur	
AX	8
CX, DX	ordonnées minima, maxima

Les valeurs de CX et DX sont permutées si DX<CX

INT 33H/9

Définition de la forme du curseur en mode graphique	
AX	9
ES:DX	adresse de la table des masques
BX,CX	abscisse, ordonnée du point de saisie

INT 33H/0AH

Définition de la forme du curseur en mode texte	
AX	0AH
BX	1 si curseur matériel 0 si curseur logiciel
CX	ligne de départ curseur (si BX=1) attribut et code du caractère (si BX=0) pour le masque d'écran : CH, CL = attribut, code
DX	ligne de fin curseur (si BX=1) attribut et code du caractère (si BX=0) pour le masque de curseur : DH, DL = attribut, code

INT 33H/0BH

Lecture du déplacement du curseur en nombre de déplacements élémentaires du curseur	
AX	0BH
CX	déplacement horizontal depuis le dernier appel de la fonction
DX	déplacement vertical depuis le dernier appel de la fonction

Le compteur est remis à zéro après appel de cette fonction

INT 33H/0CH

Définition de l'adresse de traitement des évènements	
AX	0CH
CX	masque d'appel des évènements : *bit* 0 modification de position du curseur *bit* 1 bouton gauche appuyé

	bit 2 bouton gauche relaché *bit* 3 bouton droit appuyé *bit* 4 bouton droit relaché *bit* 5 bouton milieu appuyé *bit* 6 bouton milieu relaché *bit* 7 à 15 non utilisés
ES:DX	adresse du programme de traitement de l'évènement

INT 33H fonct. 0DH/0EH

Contrôle d'émulation *light pen*	
AX	0DH : validation émulation 0EH : inhibition émulation

Lors de l'initialisation, une validation est effectuée.

INT 33H/0FH

Définition de la sensibilité de la souris	
AX	0FH
CX	rapport déplacement horizontal / nombre de pixels (défaut 8)
DX	rapport déplacement vertical / nombre de pixels (défaut 16)

INT 33H/10H

Effacement automatique de curseur	
AX	10H
CX,DX	abscisse, ordonnée point haut-gauche
SI, DI	abscisse, ordonnée point bas-droit

Si le curseur disparaît, il faut le réafficher par la fonction 1.
Certains *drivers* fournissent les coordonnées dans un tampon dont l'adresse est passée en paramètre dans ES:DX.

INT 33H/13H

Valeur du seuil de doublement de la vitesse du curseur	
AX	13H
DX	seuil au-delà duquel la vitesse est doublée (par défaut 64 déplacements élémentaires de la souris par seconde)

Si on donne une valeur irréalisable, cette option est désactivée.

INT 33H/14H

Changement de routine de traitement	
AX	14H
ES:DX	adresse de la nouvelle routine
CX	masque du nouvel appel
ES:DX	adresse de l'ancienne routine
CX	masque de l'ancien appel

Bits du masque :	– 0 le curseur change de position – 1 le bouton gauche est enfoncé – 2 le bouton gauche est relâché – 3 le bouton droit est enfoncé – 4 le bouton droit est relâché

Lorsque la routine est appelée, elle passe les informations suivantes :

AX	masque de condition (le bit correspondant à l'évènement mis à 1)
BX	état du bouton
CX	abscisse
DX	ordonnée
DI	abscisse en nombre de déplacements élémentaires
SI	ordonnée en nombre de déplacements élémentaires
DS	contient la valeur du segment du driver de la souris. La routine doit mettre à jour DS.

INT 33H/15H

Informations sur le tampon du *driver* de la souris	
AX	15H
BX	taille du tampon utilisé
Utilisé en liaison avec les fonctions 16H et 17H.	

INT 33H/16H

Sauver l'état du *driver* de la souris	
AX	16H
ES:DX	pointeur vers le tampon utilisé

INT 33H/17H

Restituer l'état du *driver* de la souris	
AX	17H
ES:DX	pointeur vers le tampon utilisé

INT 33H/1DH

Modifie le numéro de page écran	
AX	1DH
BX	numéro de page utilisé

INT 33H/1EH

Lecture du numéro de page écran	
AX	1EH
BX	numéro de page utilisé